

Desenvolvimento de Software 3

SEMANA 1 - Revisão: “Desenvolvimento de Software 2”



“

The only way to go fast is to go well.

- Robert C. Martin (Uncle Bob)



Bem-vindo!

Sou José Paulo Rodrigues.

- Mestre em Ciências pelo Programa de Pós Graduação em Sistemas de Informação da EACH/USP.
- Bacharel em Sistemas de Informação pela EACH/USP.
- Desenvolvedor de software com experiência em gestão de equipes de desenvolvimento, gestão de projetos e análise.
- Sou Engenheiro de Software na BairesDev.



Estrutura do Curso

Semana	Tópicos
Semana 1	Revisão: “Desenvolvimento de Software 2”
Semana 2	Integração de Sistemas
Semana 3	API Parte 1 – Fundamentos
Semana 4	API Parte 2 – Arquitetura
Semana 5	API Parte 3 – Segurança
Semana 6	API Parte 4 – Documentação
Semana 7	API Parte 5 – Evidências
Semana 8	Migração de Dados
Semana 9	Revisão Geral (SDLC)
Semana 10	Apresentação do Projeto Final

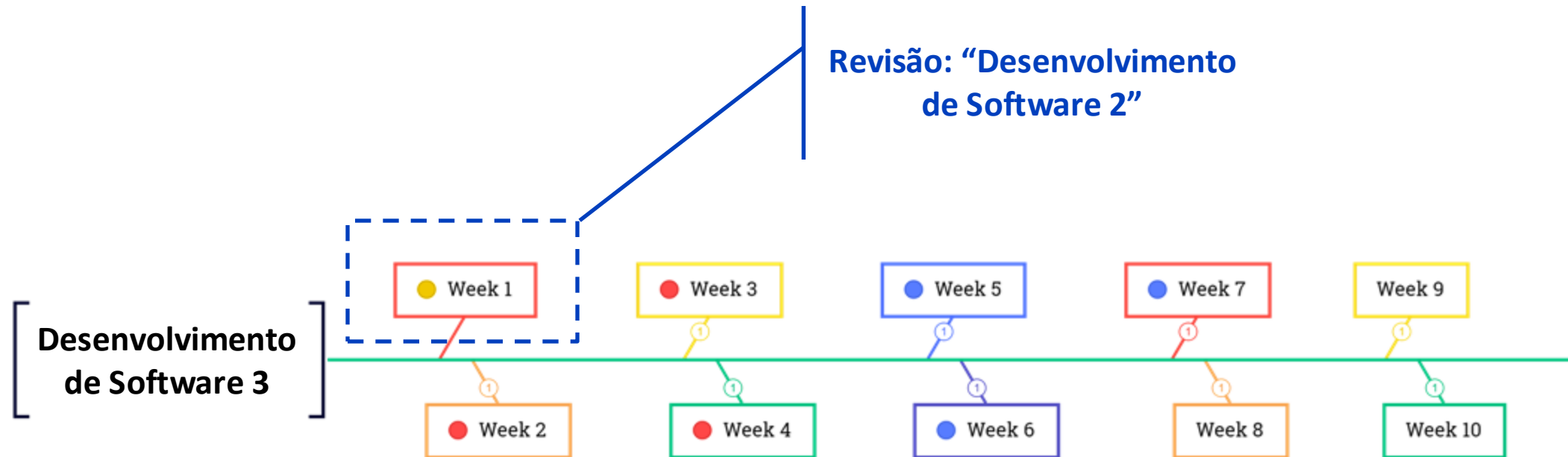


Agenda

- Revisão de Desenvolvimento de Software 2
 - Os 4P's em Planejamento de Projeto de Software
 - O Ciclo de Vida do Desenvolvimento de Software
 - Análise de Requerimentos
 - Processamento do Backlog
 - Estratégias de Estimativa
 - Monitoramento do Progresso do Trabalho
 - Boas Práticas de Codificação
 - Projeto de Sistemas
 - Testes de Desenvolvimento
 - Documentação de Projetos
 - CI/CD
 - Apresentação de Projetos
- Introdução ao Projeto Final



Conteúdo da Semana 1



Os 4P's em Planejamento de Projeto de Software

Os 4P's em Planejamento de Projeto de software



Qual é a importância de entender os 4P's no desenvolvimento de software e como influenciam no planejamento e execução dos projetos?





O Produto

Entender o Produto que se está desenvolvendo permite:

- Definir claramente requerimentos e características do software.
- Estabelecer objetivos em funcionalidade, desempenho, segurança e usabilidade.
- Compreender as necessidades do cliente e usuários finais para atender suas demandas.





O Processo

Entender a necessidade do processo de desenvolvimento permite:

- Selecionar a metodologia de desenvolvimento (Scrum, Kanban, etc.);
- Planejar iterações, atribuir tarefas, gerenciar mudanças e garantir qualidade.
- Um processo sólido contribui para a eficiência do projeto.





O Projeto

A gestão adequada do projeto de Desenvolvimento de software permite:

- Estabelecer o cronograma, definir objetivos, alocar recursos no orçamento.
- Gerencie riscos e comunique-se de forma eficaz com todas as partes interessadas.
- Uma gestão adequada garante o progresso e cumprimento dos objetivos de desenvolvimento.





As Pessoas

Entender quem são as pessoas envolvidas no projeto permite:

- Conhecer quem são os atores envolvidos na equipe e quais são seus papéis.
- Entender as habilidades, experiências e expectativas da equipe de desenvolvimento, stakeholders e usuários finais .
- Atribuir tarefas e responsabilidades adequadamente.



O Ciclo de Vida do Desenvolvimento de Software

O Ciclo de Vida de Desenvolvimento de Software



Quais são os desafios e riscos associados à falta de um Ciclo de Vida de Desenvolvimento de Software (SDLC) definido em um projeto?



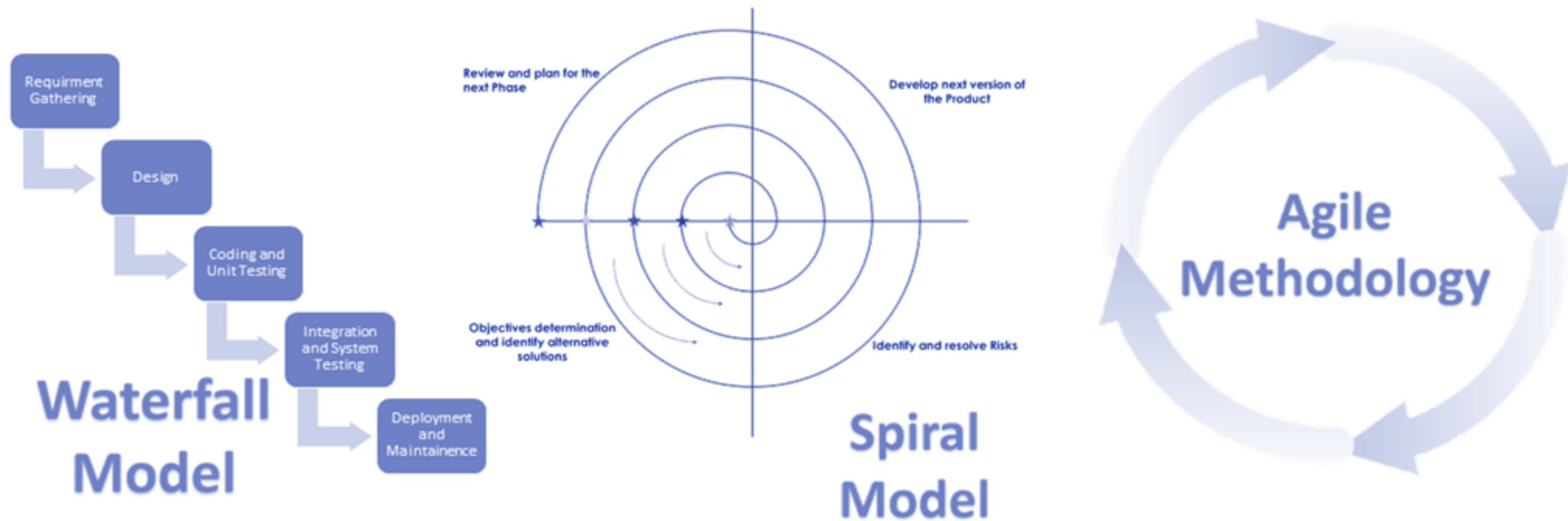


Fases do Ciclo de Vida de Desenvolvimento de Software





Modelos do SDLC





Metodologia Ágil

Qual metodologia ágil você recomendaria para um projeto de desenvolvimento de software e por quê?



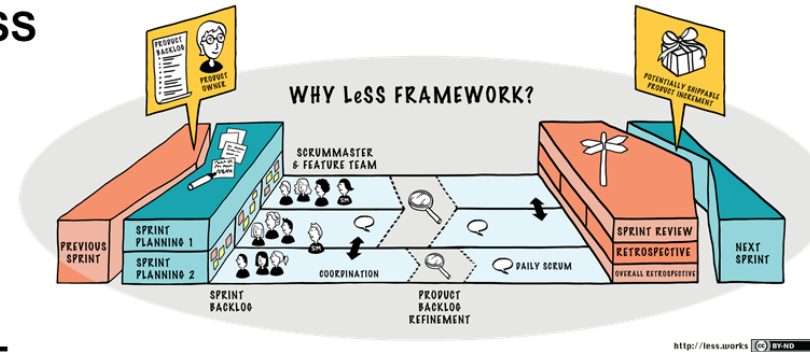


Metodologia Ágil

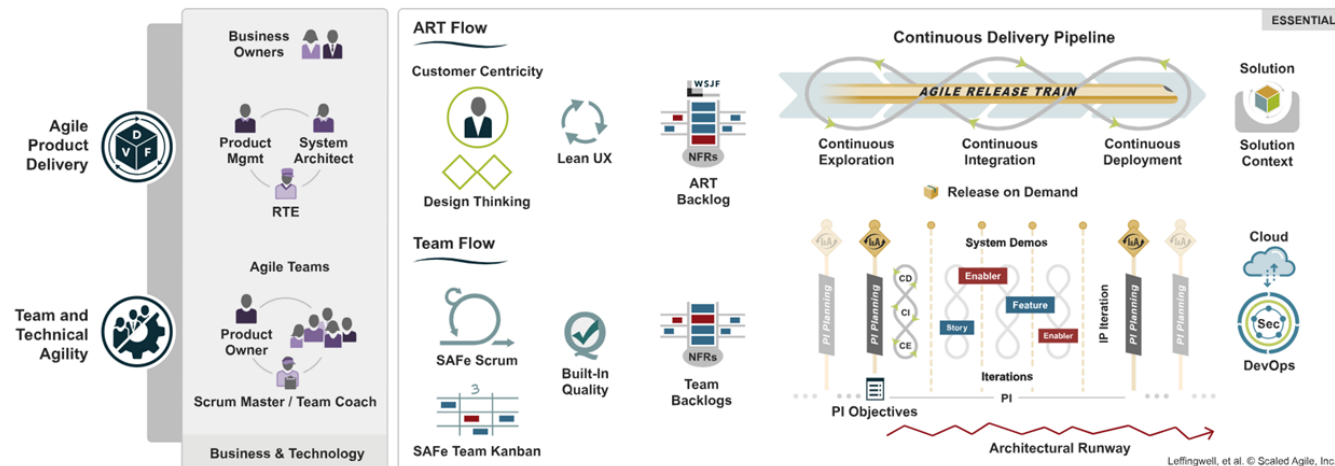
Kanban



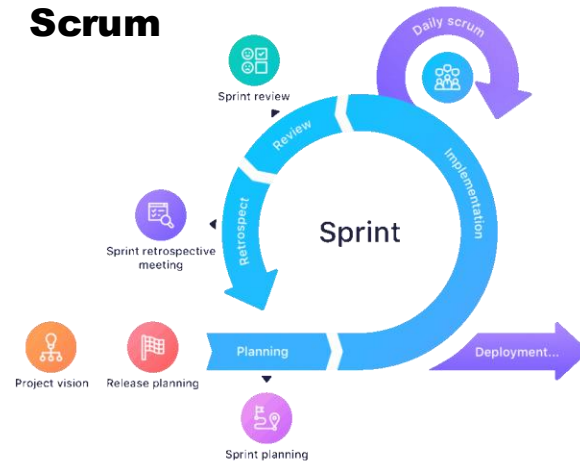
LeSS



SAFe



Scrum



Análise de Requerimentos



Metodologia Ágil

**Qual é a importância da análise de
requisitos no desenvolvimento de
software?**





A metáfora do Projeto do Balanço



Foi assim que o cliente explicou



O que o líder do projeto entendeu



Foi assim que o analista projetou



O que o programador desenvolveu



Foi assim que o consultor de negócios descreveu



Foi assim que o projeto foi documentado



Foi assim que foi instalado



O que foi cobrado do cliente



Essa foi a manutenção



O que o cliente realmente precisava



Requerimentos Funcionais e Não-Funcionais

Requerimentos funcionais:

- Os requerimentos funcionais descrevem o que o software pode fazer.
- Eles são importantes para entender como o software deve funcionar.
- Eles ajudam a garantir que o software faça o que é esperado.





Requerimentos Funcionais e Não-Funcionais

Requerimentos Não-funcionais:

- Incluem itens como desempenho, segurança, usabilidade, escalabilidade, disponibilidade e manutenibilidade.
- Eles definem como o software deve se comportar além de suas funcionalidades básicas.
- Embora não estejam diretamente relacionados com ações específicas, são importantes para a confiabilidade, eficiência e qualidade do sistema.





Análise de Requerimentos

Quais técnicas você utiliza para analisar requisitos em projetos de software e por quê?



Casos de Uso em análise de requerimentos



Casos de Uso **descrevem como interagem os usuários e o sistema**, capturando requisitos e comunicando expectativas. Também **validam requisitos, melhoram a usabilidade e orientam o design a partir da perspectiva do usuário**. Além do mais, servem de base para o projeto técnico, testes, controle de mudanças, avaliação de riscos, estimativa e planejamento do projeto. **Os diagramas de casos de uso visualizam essas interações de uma forma compreensível.**

Compra de Combustível

1. O Cliente inicia **“Comprar Combustível”** com os dados do seu cartão de crédito (incluindo CEP).
2. O sistema pede o CEP para validar (combinar com o cartão de crédito para confirmar a propriedade).
3. Caso haja correspondência, o sistema acessa **“Validar Crédito e Gerar Fatura”** e verifica o cartão na Agência de Crédito (histórico bom, não roubado).
4. Depois de Validado pela Agência de Crédito, o sistema volta para a opção **“Comprar Combustível”**.
5. O sistema solicita ao Cliente que escolha o tipo de gasolina e carregue o veículo.
6. O Cliente carrega o veículo e pára a bomba quando termina.
7. O sistema retorna para **“Validar Crédito e Gerar Fatura”** para finalizar a transação e imprimir o recibo (Nota Fiscal).

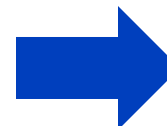
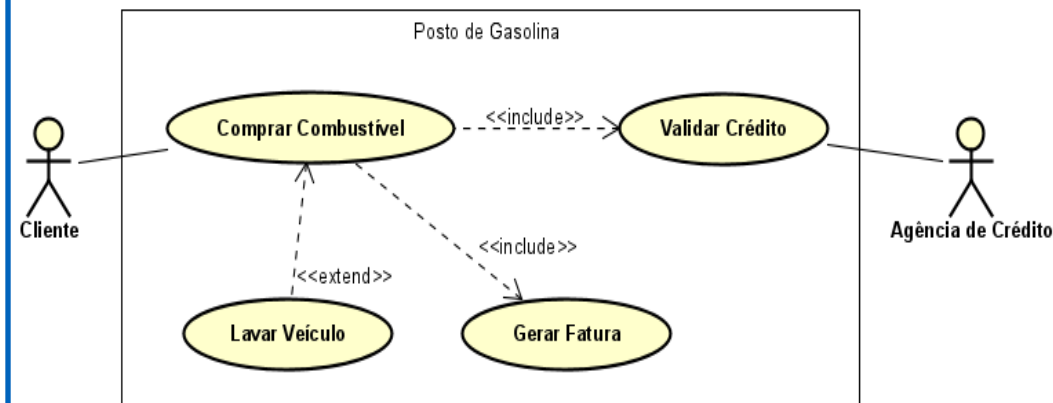
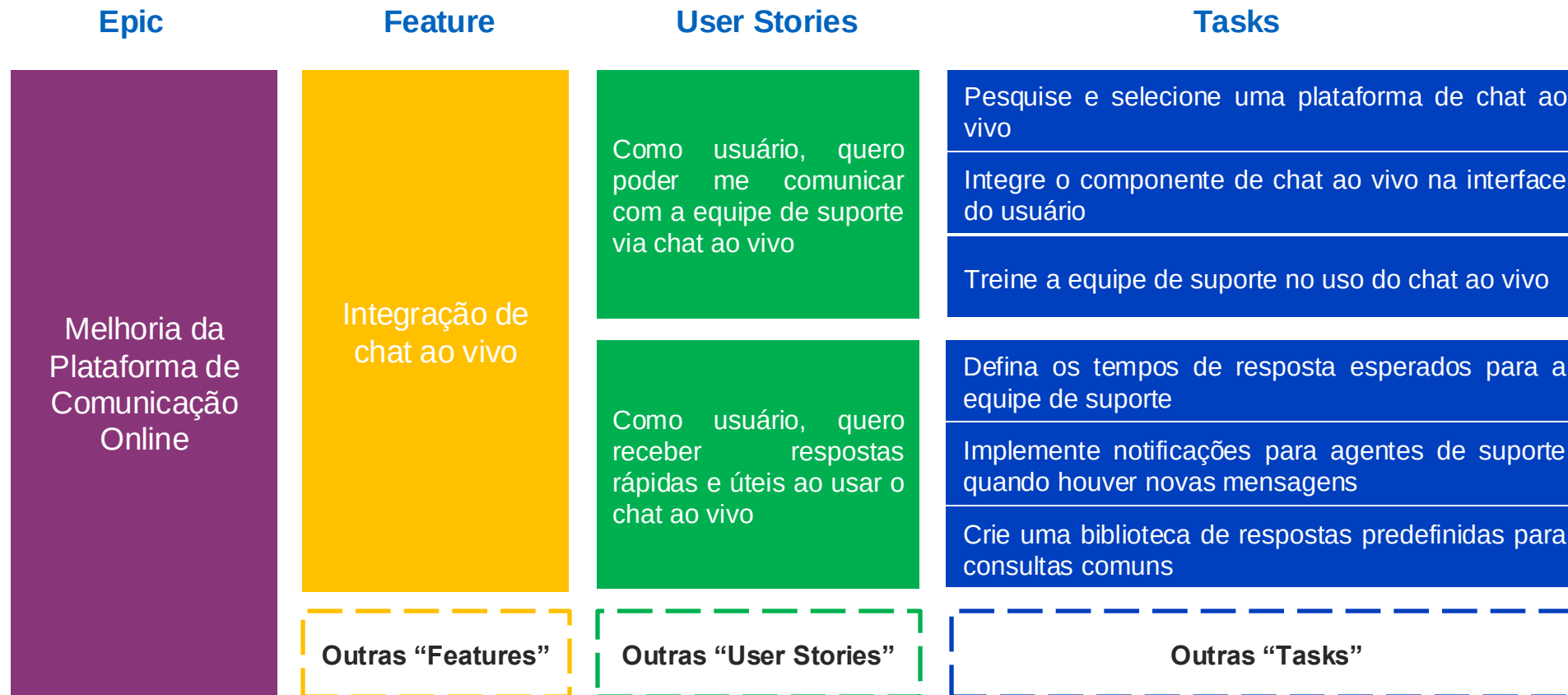


Diagrama de caso de uso de uma empresa de posto de gasolina descrevendo o cenário de compra de combustível.





Níveis de trabalho no Desenvolvimento de Software ágil





A “Definição de Pronto”

DoR “Definition of Ready” (Definição de Pronto):

- Estabelece critérios para os itens do backlog antes do sprint.
- Garante clareza e preparação para a equipe.
- Promove transparência e eficiência.
- Os critérios são flexíveis e adaptáveis.

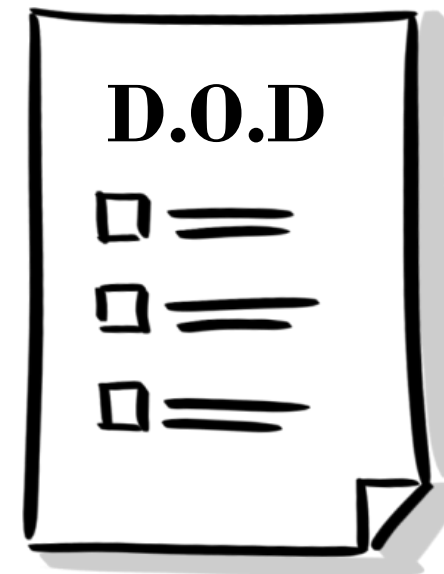




A “Definição de Feito”

DoD “Definition of Done” (Definição de Feito):

- Define os critérios para concluir um item do backlog ao finalizar um sprint.
- Estabelece padrões de qualidade e funcionalidade acordados pela equipe.
- Garante que as entregas sejam consistentes e verificáveis.
- Prepara os itens para implantação ou entrega ao cliente no final do sprint.

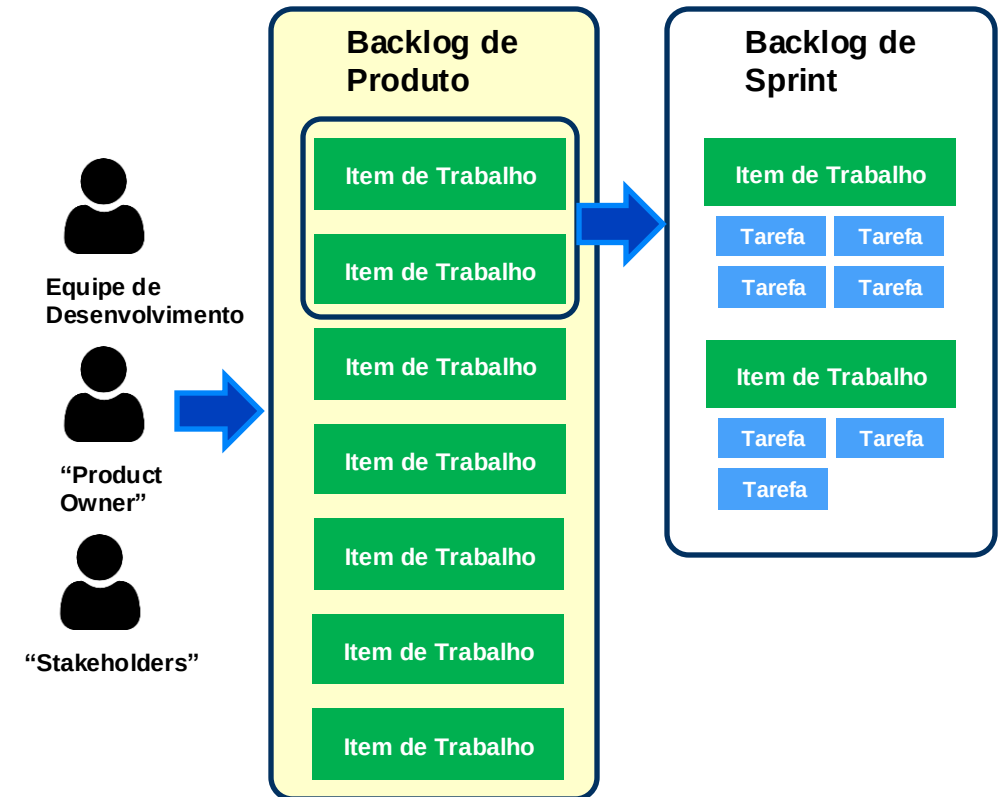


Processamento do Backlog



O Backlog do Produto

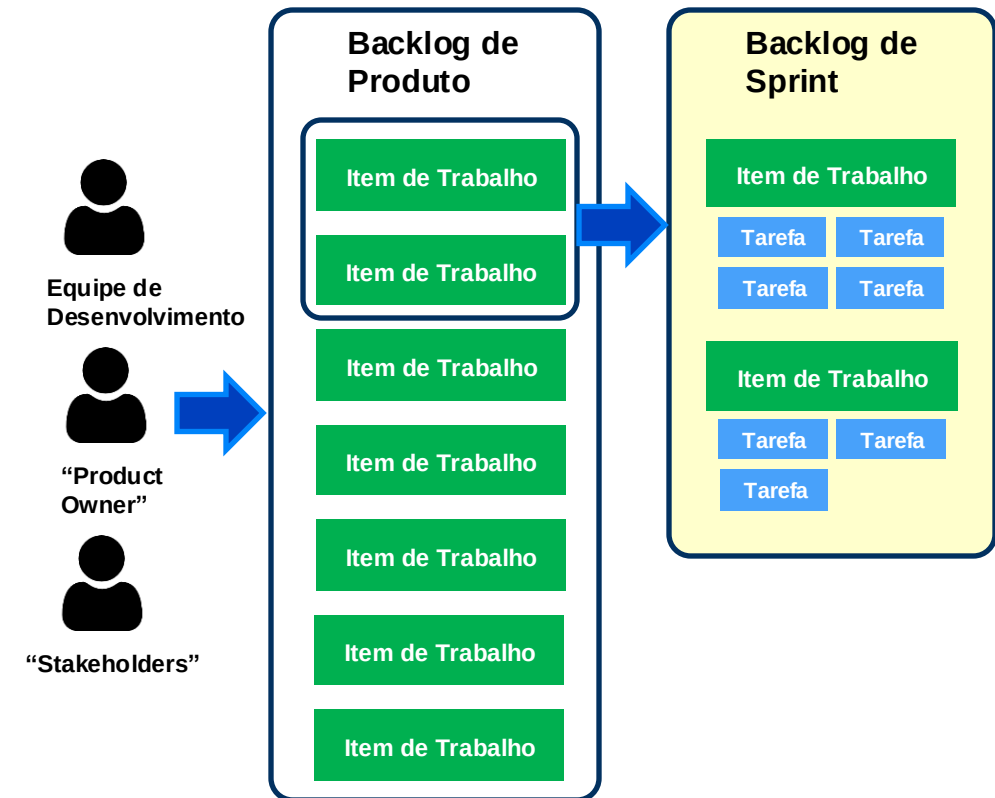
- Em Scrum, o "backlog" é **uma lista estruturada e priorizada** de PBIs (Product Backlog Items).
- Os PBIs geralmente representam **histórias de usuários**, mas também podem incluir recursos, melhorias, correções de bugs, pesquisas e outros itens.
- O “backlog” **contribui para o processo de desenvolvimento do produto ao fornecer uma visão clara** das tarefas pendentes e sua prioridade.





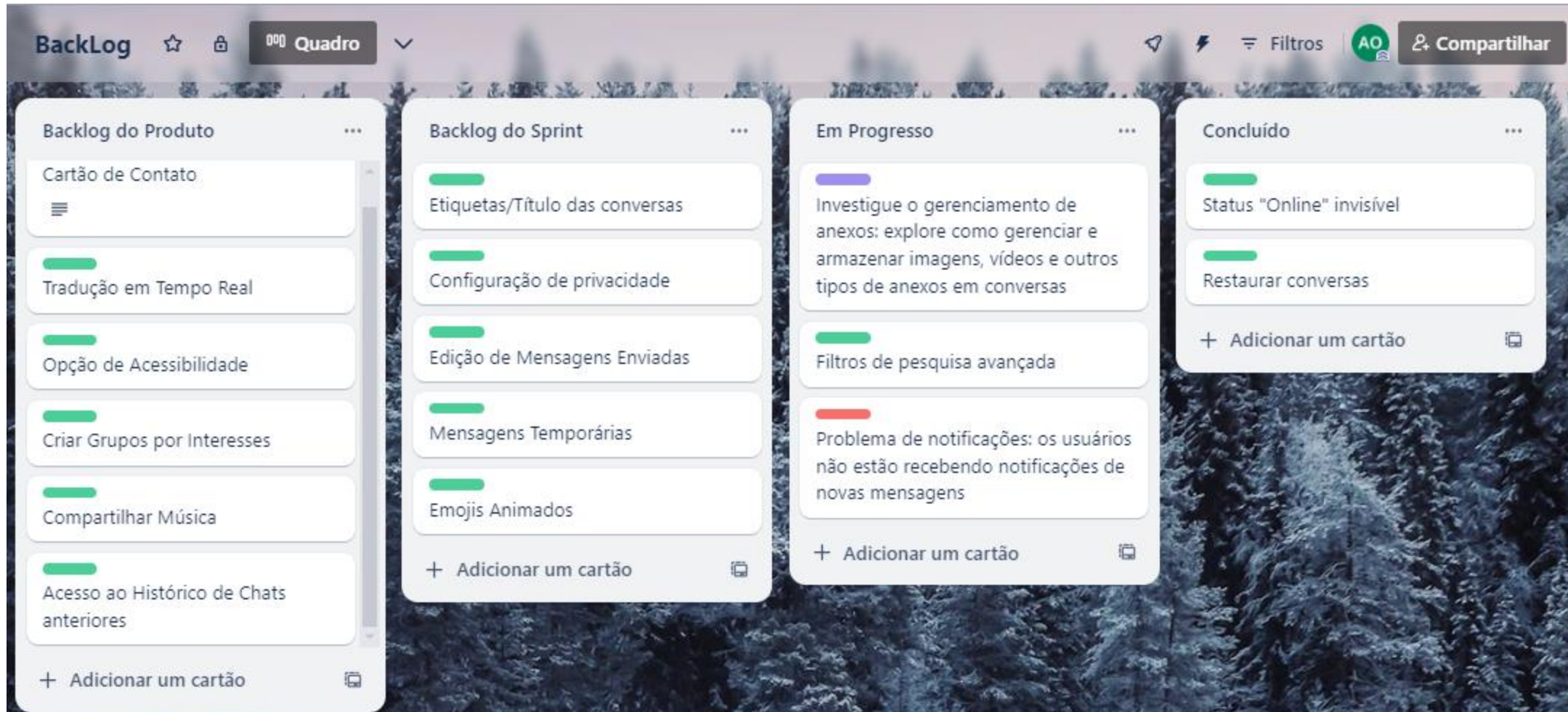
O Backlog do Sprint

- O Backlog do Sprint é uma lista detallada de itens de trabalho extraídos do Backlog do Produto.
- A equipe se compromete em concluir estes itens durante um sprint específico.
- Durante a cerimônia de planejamento do sprint, a equipe seleciona os PBIs considerados alcançáveis e os adiciona ao Backlog do Sprint.
- Os itens do Backlog do Sprint são divididos em tarefas específicas e detalhadas para implementação durante o sprint.





A representação visual do backlog



Estratégias de Estimativa



Estimativas em Alto nível

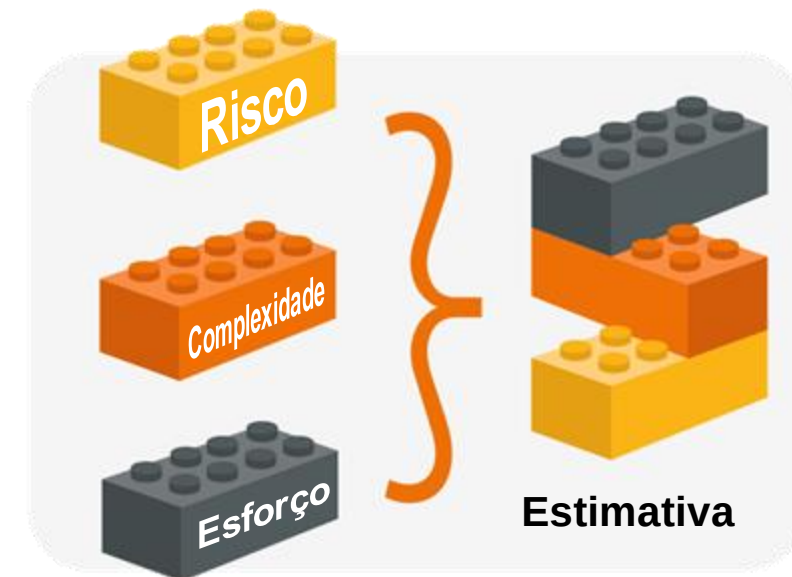
- Elas proporcionam uma visão panorâmica do início do projeto.
- Facilitam a priorização ou oferecem uma ideia geral da carga de trabalho e dos recursos necessários.
- Elas avaliam o esforço, a complexidade e a magnitude relativa das tarefas.
- Para simplificar a complexidade, por exemplo, “tamanhos de camisetas” podem ser usados em vez de valores numéricos específicos.
- Elas servem como um guia inicial e são ajustados à medida que as tarefas reais são melhor compreendidas.





Estimativas dos itens do Backlog

- A estimativa de PBIs é importante no Scrum para planejar o trabalho em cada Sprint.
- São atribuídos valores que representam a complexidade e o esforço de cada item do backlog (story points).
- Um backlog bem refinado e estimado simplifica o planejamento e a execução do projeto.
- Permite tomar decisões sobre como abordar o trabalho em cada iteração.



<https://aginic.com/blog/agile-estimation>

A técnica de estimação usando o “Scrum Poker”



- Durante esta prática, os membros da equipe avaliam o esforço necessário para concluir uma história de usuário.
- Cada membro escolhe uma carta que representa a sua percepção de esforço de forma privada.
- As estimativas são então reveladas simultaneamente e discutidas em caso de discrepâncias significativas.
- A equipe trabalha em conjunto para chegar a um consenso sobre a estimativa final, registrando-a no Backlog do Produto.



<https://www.easyagile.com/blog/planning-poker/>

A técnica de estimava usando o “Scrum Poker”



Como você descreveria a relação entre a sequência de Fibonacci e o esforço que uma história de usuário representa durante o processo de estimativa?



Monitoramento do Progresso de Trabalho

Monitoramento do Progresso de Trabalho



Que técnicas você usa para medir o progresso e o alcance das metas?





O Plano de Ação (Roadmap) no Projeto

- O "Roadmap" é uma representação visual e estratégica do plano de ação para um produto.
- Comunica a visão geral, metas e etapas chaves de desenvolvimento de maneira compreensível.
- Proporciona a estrutura para o planejamento estratégico no desenvolvimento ágil.
- Permite ajustes e adaptações de acordo com a evolução do projeto.
- Identifica dependências entre itens e as fases do projeto.
- Facilita a priorização informada de recursos e melhorias com base em sua importância e valor.

Exemplo de um Plano de Ação para o Desenvolvimento de uma Loja Online

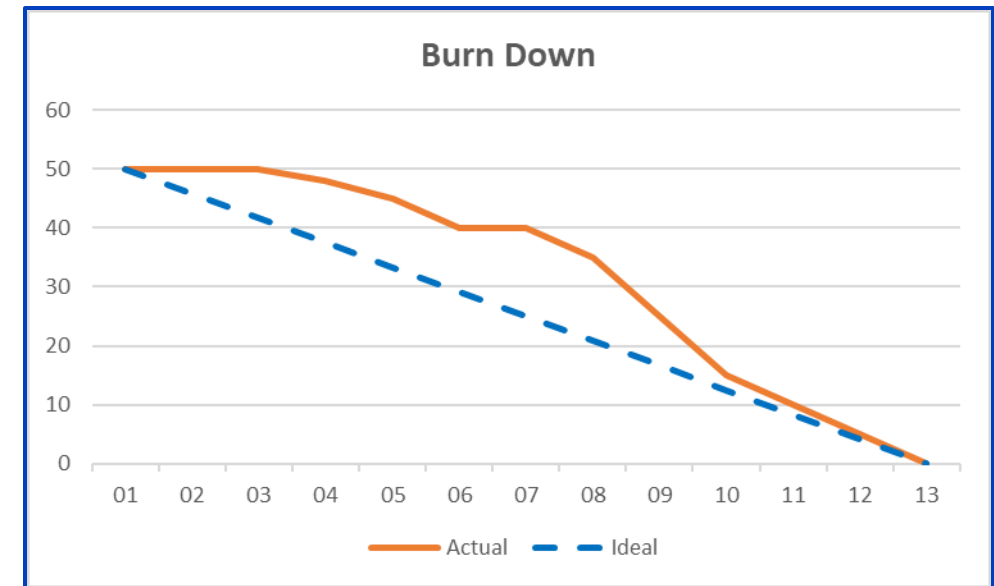
1º Trimestre			2º Trimestre			3º Trimestre		
Jan	Fev	Mar	Abr	Mai	Jun	Jul	Ago	Set
Configuração Básica da Loja								
	Catálogo de Produtos e Carrinho de Compras							
		Processo de Pagamento e Faturamento						
				Otimização de Buscas e Filtros				
					Integração com Redes Sociais			
							Integração com Redes Sociais	
							Gestão de comentários e avaliações	





O “Burn Down”

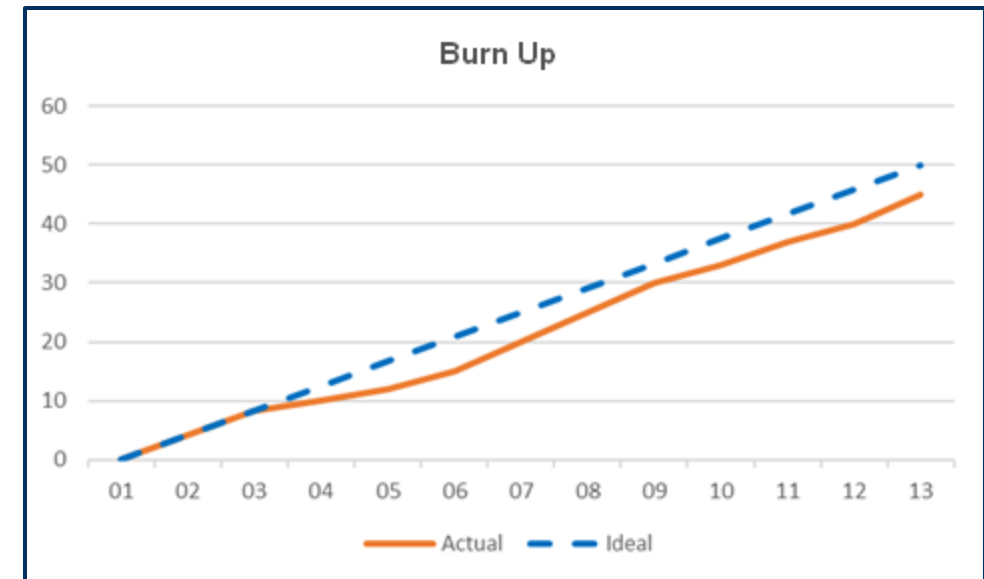
- O "Burn Down" é uma ferramenta que visualiza a evolução do trabalho pendente durante um sprint.
- No gráfico, o eixo vertical representa a quantidade de trabalho pendente, geralmente em pontos de história.
- O eixo horizontal mostra o progresso ao longo do tempo, dividido em intervalos como dias, semanas ou sprints.





O “Burn Up”

- O "Burn Up", conhecido como “Gráfico de Aumento”, concentra-se no trabalho concluído.
- No gráfico, o eixo vertical representa a quantidade total de trabalho ou escopo, geralmente em unidades como pontos de história, histórias de usuários ou recursos.
- O eixo horizontal representa o tempo, dividido em intervalos como dias, semanas ou sprints.



Boas Práticas de Codificação



Boas práticas de codificação

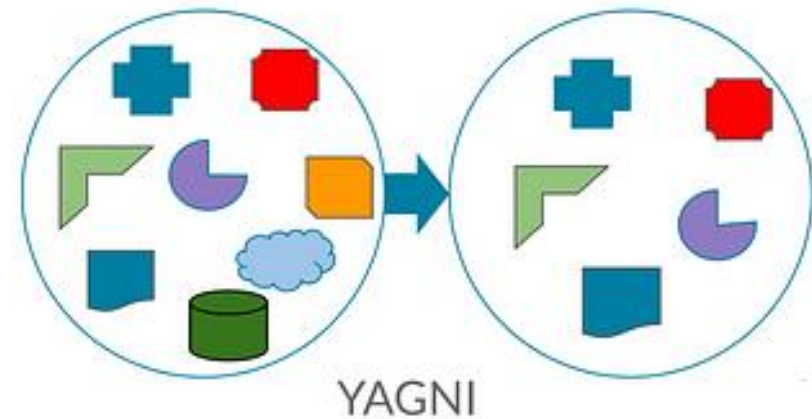
Quais são as boas práticas e princípios de codificação que você implementa no desenvolvimento de projetos de software?





O princípio “YAGNI”

- YAGNI significa "You Ain't Gonna Need It" (você não vai precisar disso) É um princípio de desenvolvimento de software.
- Sugere evitar a implementação de funcionalidades não necessárias no momento.
- Evita introduzir complexidade desnecessária no código.
- Tem como foco implementar apenas as funcionalidades necessárias para atender aos requisitos atuais do projeto.
- Ajuda a manter o código simples, flexível e fácil de manter.

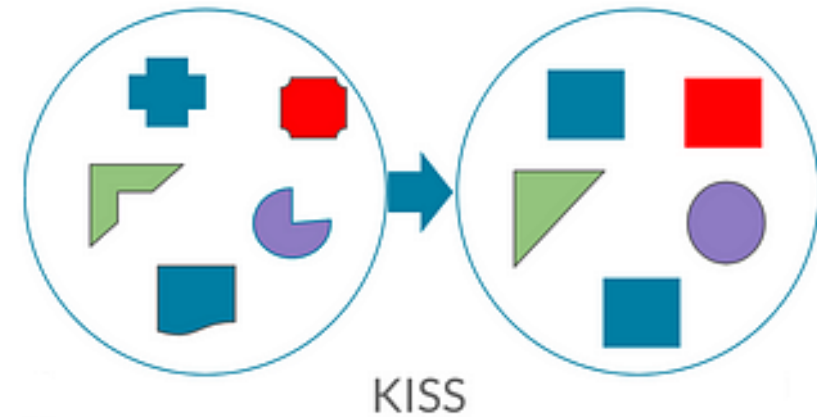


<https://levelup.gitconnected.com/demystifying-software-development-principles-dry-kiss-eagni-solid-grasp-and-lod-8606113c0313>



O princípio “KISS”

- KISS, "Keep It Simple, Stupid" (Mantenha isso simples, idiota), promove soluções simples e diretas.
- Elimina a complexidade desnecessária para melhorar a compreensão e a eficiência.
- Procura equilibrar simplicidade e funcionalidade no design e desenvolvimento.
- Promove clareza e facilidade de uso em produtos e sistemas.
- Reduz a probabilidade de erros e facilita a manutenção a longo prazo.

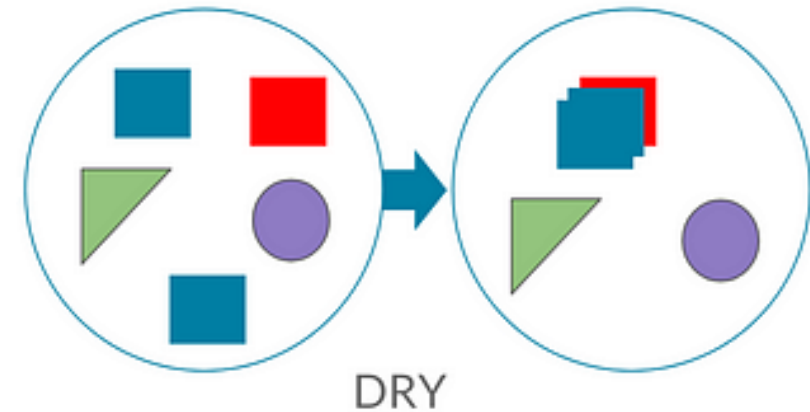


<https://levelup.gitconnected.com/demystifying-software-development-principles-dry-kiss-eagni-solid-grasp-and-lod-8606113c0313>



O princípio “DRY”

- DRY, "Don't Repeat Yourself" (Não se repita), é uma filosofia de desenvolvimento de software.
- Busca reutilizar e reduzir a duplicação de código em um projeto.
- Cada conhecimento ou lógica deve ter uma representação única no código.
- Evita escrever as mesmas informações ou lógica em vários lugares.



<https://levelup.gitconnected.com/demystifying-software-development-principles-dry-kiss-eagni-solid-grasp-and-lod-8606113c0313>

O princípio “SOC”

- SOC, "Separation of Concerns" (Separação de preocupações, responsabilidades), é um princípio no design de software.
- Divide um sistema em módulos ou componentes separados, cada um com uma responsabilidade específica.
- Busca melhorar a modularidade, legibilidade e facilidade de manutenção do código.
- Reduz a complexidade e facilita o gerenciamento da mudança para focar nas preocupações individuais.



<https://marchoeijmans.blogspot.com/2012/06/principle-of-separation-of-concerns-soc.html>

Projeto de Sistemas



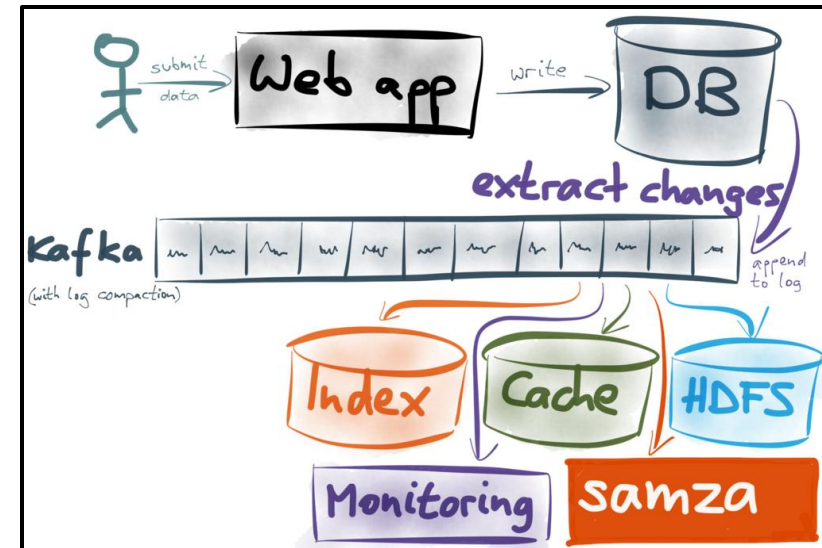
Projeto de Sistemas

Qual é a diferença entre estilos de arquitetura e padrões arquitetônicos, você pode citar exemplos?





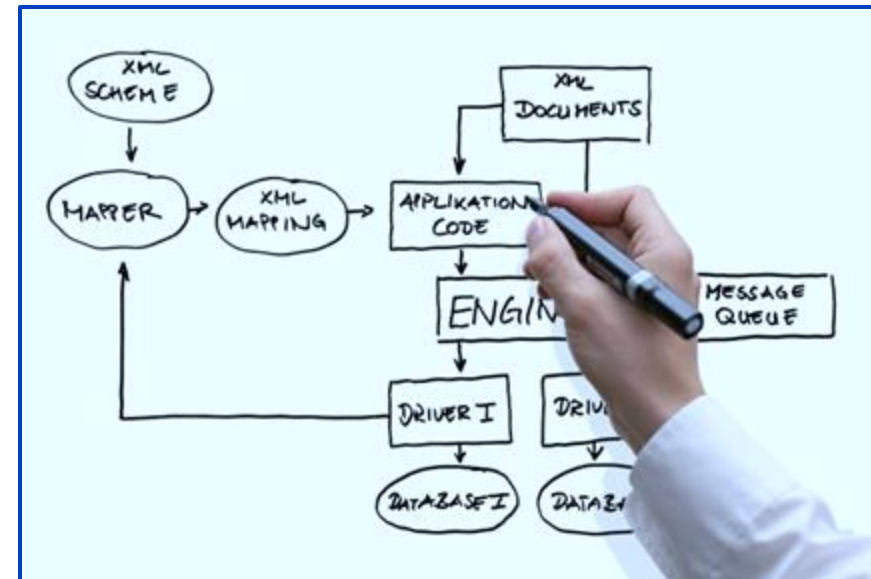
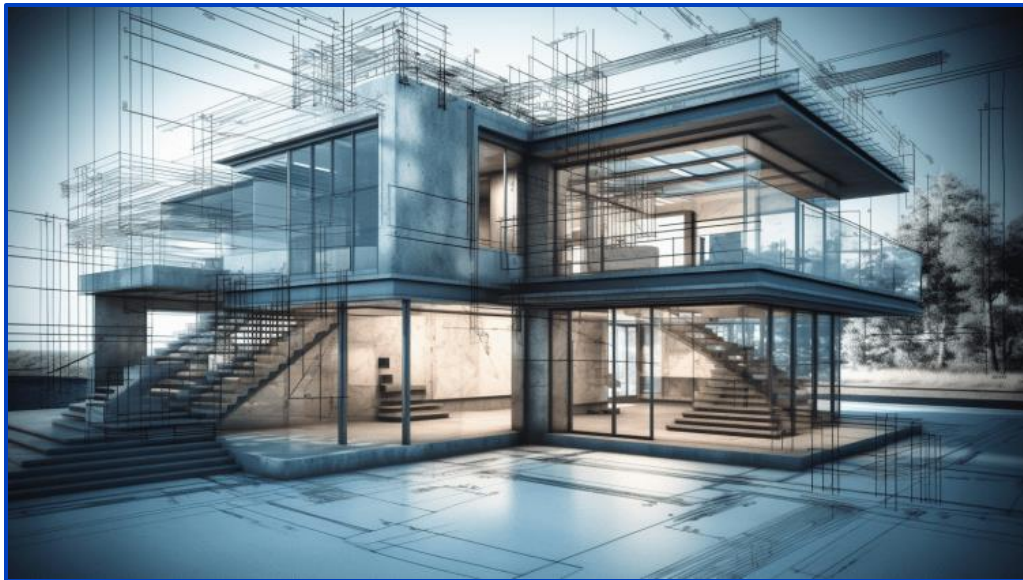
A Arquitetura de Software



A arquitetura de software, como uma receita, fornece a estrutura e a orientação para o desenvolvimento de um sistema, detalhando como os componentes se relacionam e colaboram entre si.



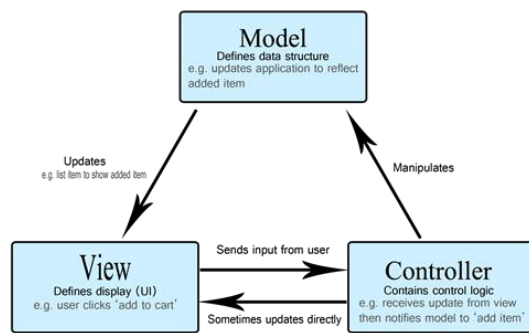
A Arquitetura de Software



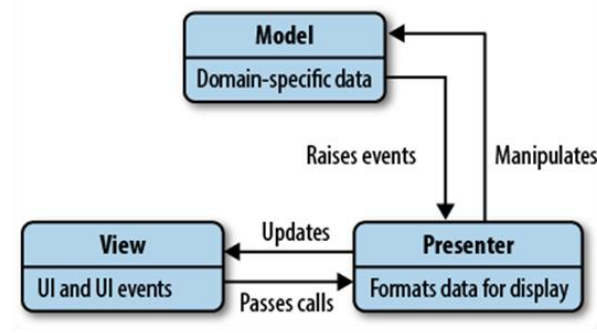
Assim como um arquiteto projeta uma casa com estrutura sólida, layout eficiente e design estético, um arquiteto de software planeja a estrutura e organização de um sistema de software.



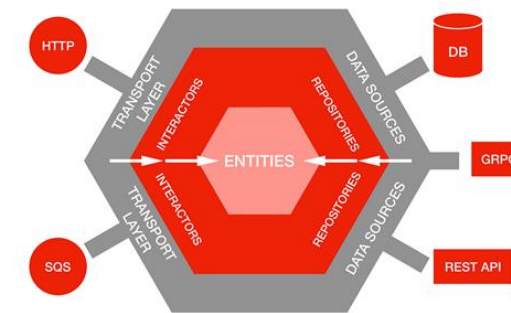
Padrões Arquitetônicos



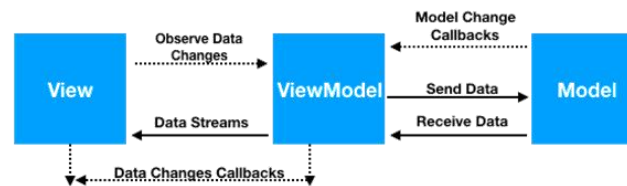
MVC (Model-View-Controller)



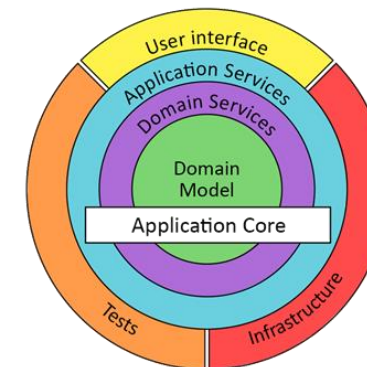
MVP (Model-View-Presenter)



Hexagonal



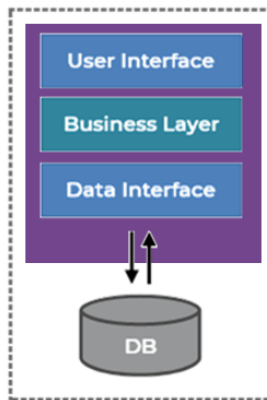
MVVM (Model-View-ViewModel)



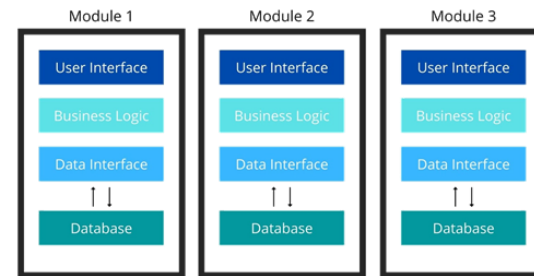
Onion



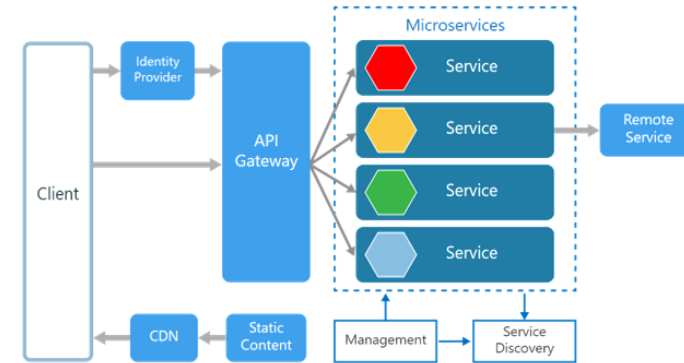
Estilos de Arquitetura



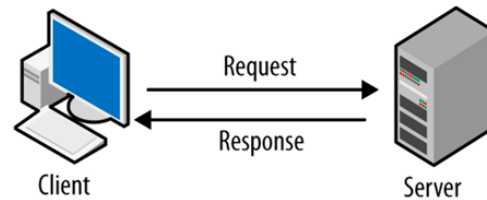
Monolith



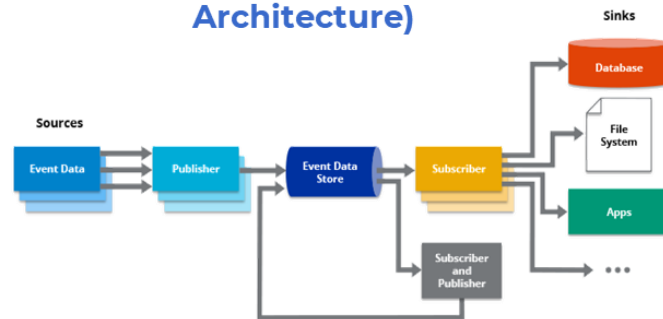
Modular Monolith



MSA (Microservices Architecture)



Client-Server Architecture



EDA (Event-Driven Architecture)



Projeto de Sistemas

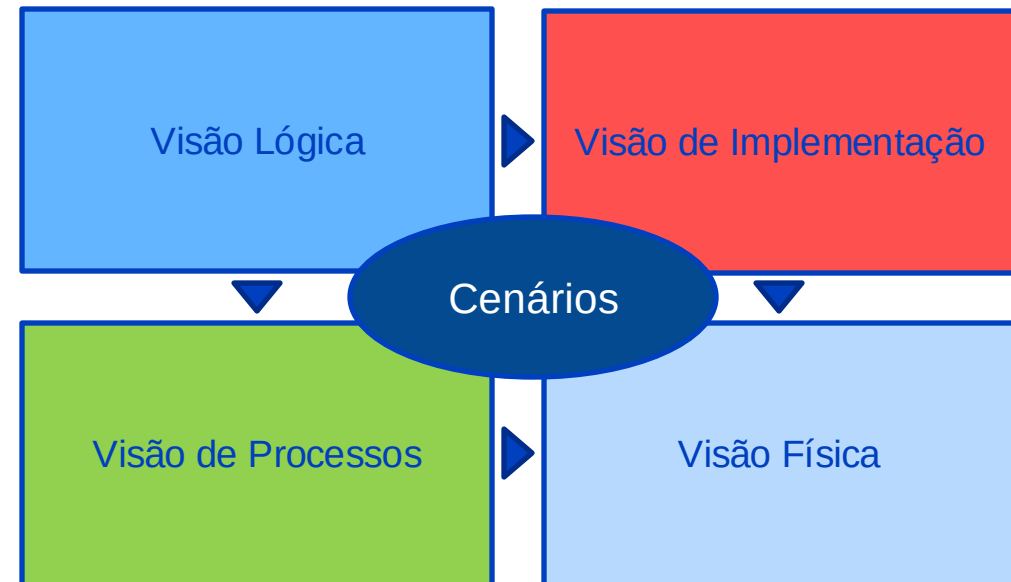
Que técnicas você conhece para representar a arquitetura de software?





O modelo de visualização 4+1

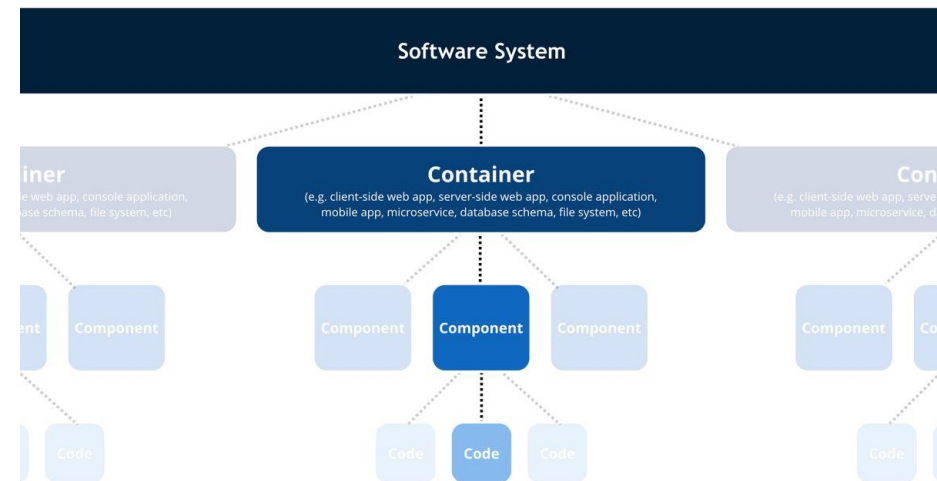
- O modelo "4+1" é uma abordagem de projeto arquitetônico de Philippe Kruchten.
- É composto por quatro visões principais: lógica, processos, implementação e física.
- Também inclui uma quinta visão, os cenários.
- Essas perspectivas combinadas oferecem uma compreensão completa da arquitetura do sistema de software.





O Modelo C4

- O modelo C4 é um conjunto de representações visuais para arquitetura de software.
- Fornece uma visualização hierárquica do sistema, dividido em Contexto, Containers, Componentes e Código.
- Não é obrigatório utilizar os quatro níveis; Somente aqueles que são relevantes devem ser usados.
- Os diagramas System Context (Contexto do Sistema) e Container podem ser adequados para a maioria das equipes de desenvolvimento.



<https://c4modo.com>

Testes de Desenvolvimento

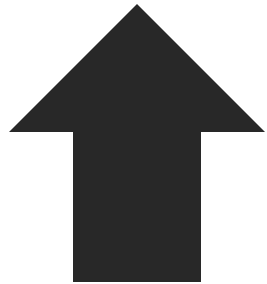


Os Testes de Desenvolvimento

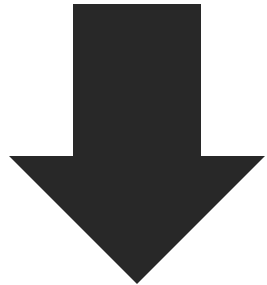
Que tipos de testes podem ser aplicados em um projeto de desenvolvimento?



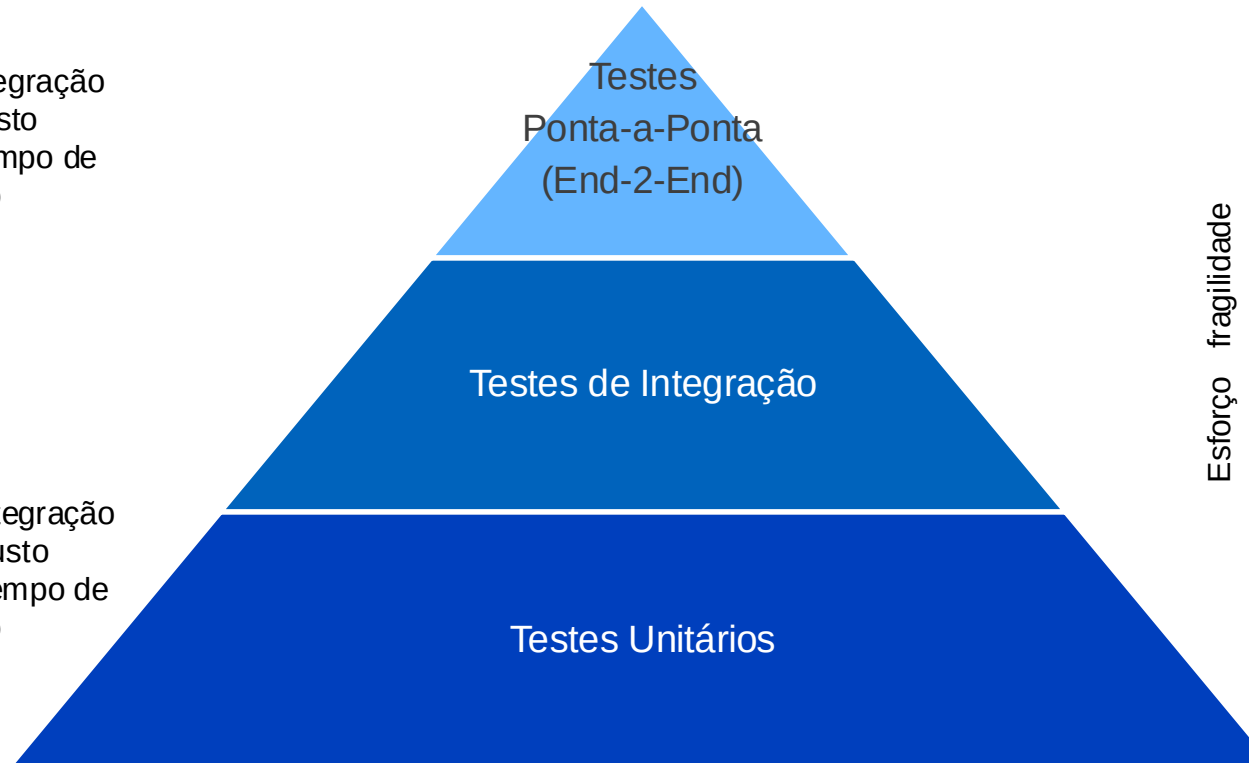
A pirâmide de testes



- Maior Integração
- Maior Custo
- Maior Tempo de execução



- Menor Integração
- Menor Custo
- Menor Tempo de execução



Esforço fragilidade

horas

Tempo para se obter
retroalimentação

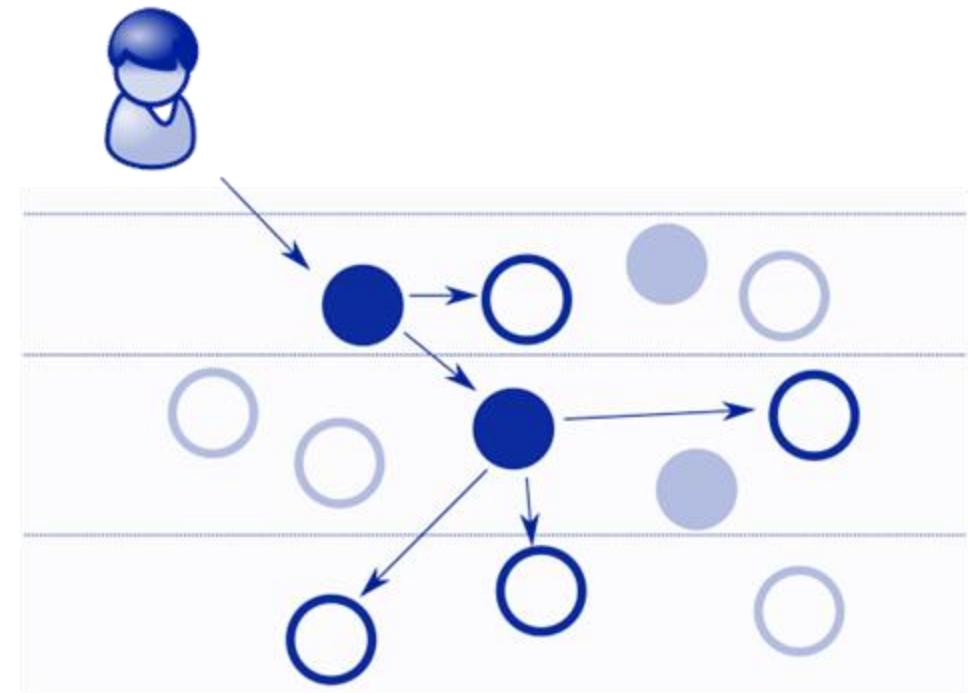
segundos



Testes End-To-End

- Os testes ponta a ponta imitam a experiência do usuário final.
- Eles verificam o funcionamento ininterrupto de todas as partes do sistema.
- Eles cobrem todo o fluxo do aplicativo.
- Eles garantem a ausência de problemas que afetem a usabilidade.
- Eles exigem um tempo considerável para serem executados.

Escopo dos Testes End-to-End



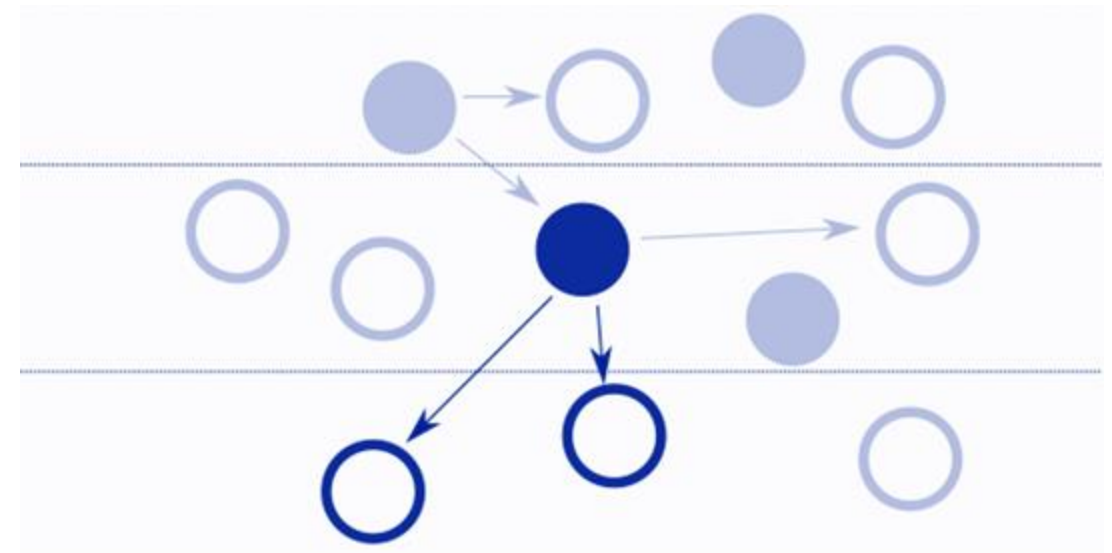
Kaczanowski, T. (2013). *Practical Unit Testing with JUnit and Mockito*. Addison-Wesley



Testes de Integração

- Os testes de integração focam na incorporação correta de módulos de código.
- Geralmente são mais rápidos que os testes ponta a ponta, mas mais lentos que os testes unitários, devido à configuração de recursos e à participação de entidades como bancos de dados.
- Para verificar os resultados, é comum examinar recursos externos, como a emissão de consultas SQL.

Escopo dos Testes de Integração



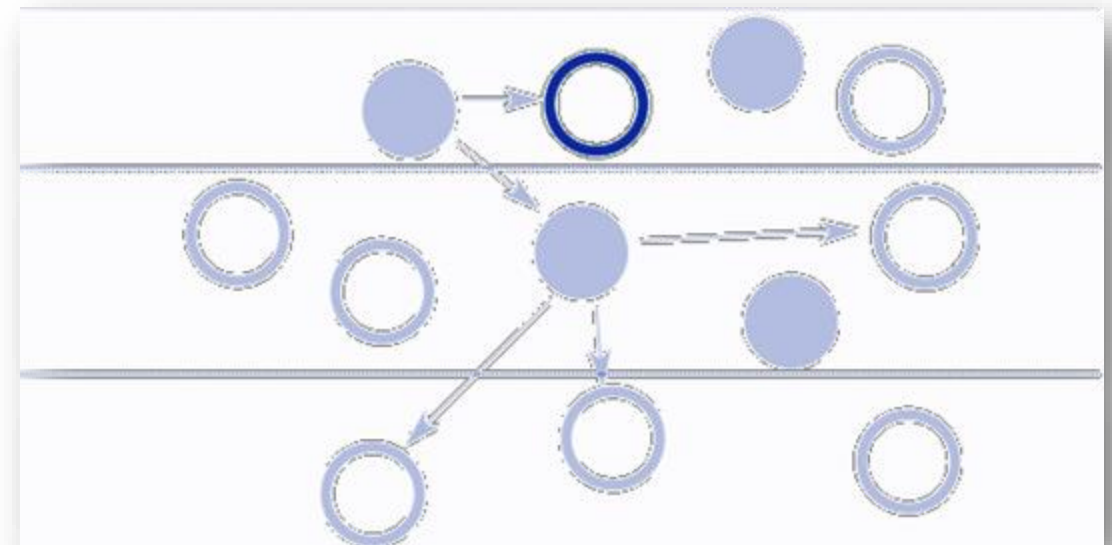
Kaczanowski, T. (2013). Practical Unit Testing with JUnit and Mockito. Addison-Wesley



Testes Unitários

- Eles avaliam unidades individuais de código, como funções ou métodos.
- São executados de forma isolada e automatizada para garantir a integridade do código.
- Eles ajudam a identificar erros e problemas lógicos nos estágios iniciais de desenvolvimento.
- Eles facilitam a refatoração e a manutenção do código ou fornecem validação contínua.

Escopo dos Testes Unitários

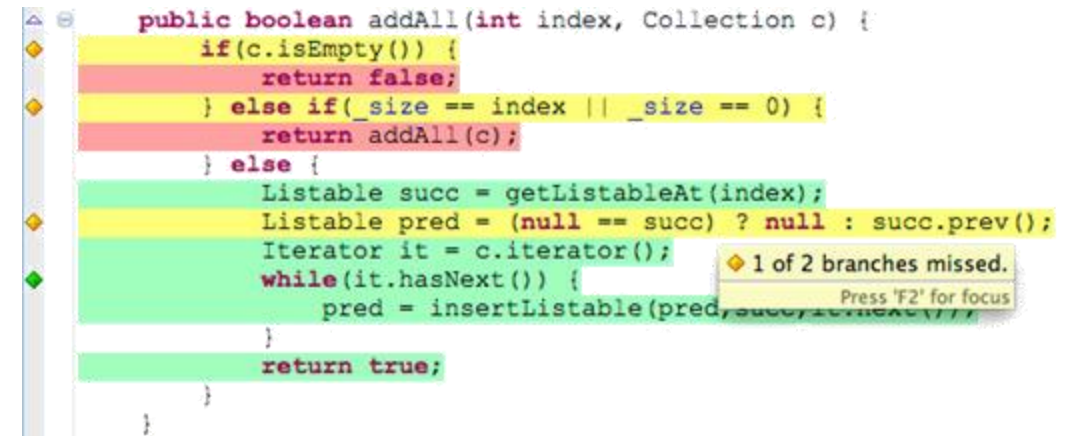


Kaczanowski, T. (2013). *Practical Unit Testing with JUnit and Mockito*. Addison-Wesley



Cobertura de Código

- A cobertura de código avalia quão bem os testes de unidade executam o código-fonte.
- É de grande ajuda medir o escopo dos testes unitários e garantir a qualidade do código.
- A alta cobertura indica que a maior parte ou todo o código foi testado, mas não garante a ausência de bugs.
- Tipos: cobertura de linha, instrução, ramificação, método e declaração.



<https://developers.redhat.com/blog/2017/10/06/java-code-coverage-eclipse>



Desenvolvimento Guiado por Testes (TDD)

- TDD envolve escrever testes antes do código.
- Ele se concentra no comportamento desejado.
- Reduza a funcionalidade ao essencial de acordo com os casos de teste.
- Garante alta cobertura de código.
- Aprender TDD e adaptar-se às suas práticas pode exigir recursos e tempo consideráveis.

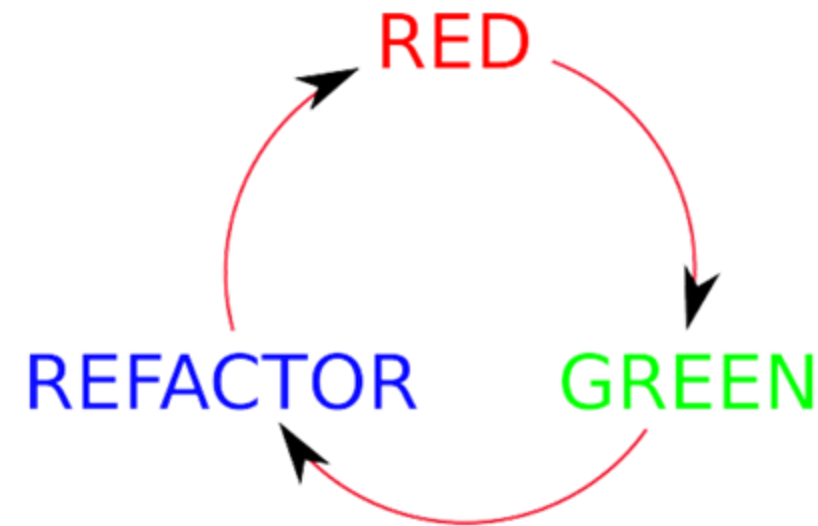


<https://marsner.com/blog/why-test-driven-development-tdd/>



O ciclo fundamental de TDD

Escreva um teste; desenvolva o código para fazê-lo funcionar; simplifique o código ao máximo para eliminar redundância. Repita este processo.



Kaczanowski, T. (2013). Practical Unit Testing with JUnit and Mockito. Addison-Wesley

Documentação de Projetos



Documentação de Projetos

Quais são algumas plataformas e técnicas de documentação que você conhece para documentar projetos?





Documentação do código fonte

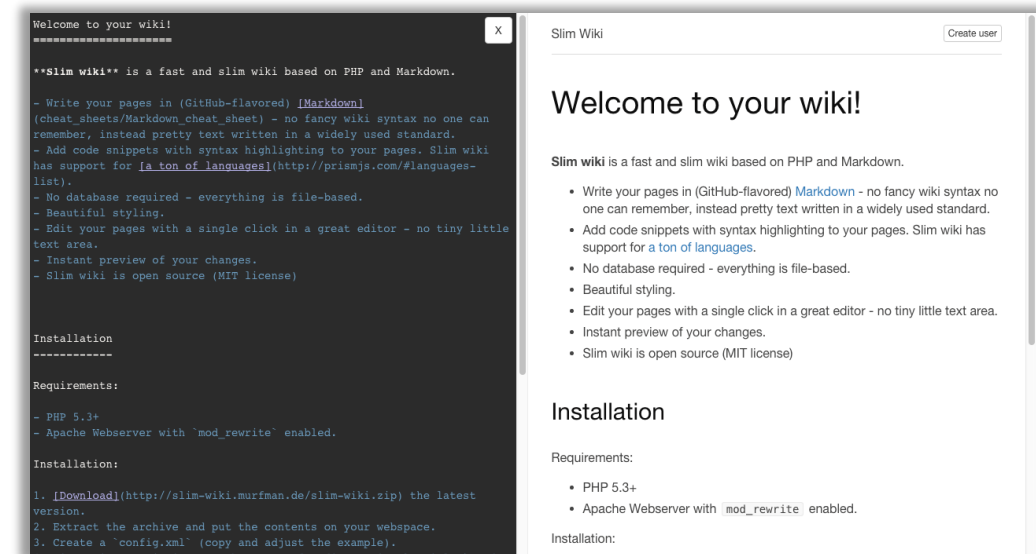
- Consiste em explicações e comentários dentro do código-fonte.
- Ele serve como um guia para desenvolvedores atuais e futuros, detalhando como o código funciona e como usá-lo de forma eficaz.
- É criado por meio de comentários no código, fornecendo informações sobre finalidade, uso e avisos.
- Melhora a legibilidade e a manutenção do código, facilitando a colaboração e a resolução de problemas.

```
1
2 public class BinarySearch {
3
4     /**
5      * Binary Search algorithm to find a target element in a sorted array.
6      * @param arr The sorted array to search.
7      * @param target The element to find.
8      * @return The index of the target element, or -1 if not found.
9      */
10    public static int binarySearch(int[] arr, int target) {
11        int left = 0, right = arr.length - 1;
12
13        while (left <= right) {
14            int mid = left + (right - left) / 2;
15            if (arr[mid] == target) return mid;
16            if (arr[mid] < target) left = mid + 1;
17            else right = mid - 1;
18        }
19
20        return -1; // Target not found
21    }
22
23    public static void main(String[] args) {
24        int[] sortedArray = {1, 3, 5, 7, 9, 11, 13, 15};
25        int target = 7;
26
27        System.out.println("Sorted Array: " + Arrays.toString(sortedArray));
28        int result = binarySearch(sortedArray, target);
29
30        System.out.println(result != -1
31            ? "Target element " + target + " found at index " + result
32            : "Target element " + target + " not found in the array.");
33    }
34 }
35
```



O Wiki do Projeto

- É uma plataforma online que permite aos usuários colaborar para criar e editar conteúdo juntos.
- Em projetos de software, facilita a colaboração na criação e manutenção de documentação fundamental, como especificações técnicas e guias do usuário.
- Melhora a comunicação e a gestão da informação, sendo crucial para o sucesso de projetos em evolução, ao fornecer um espaço central para documentar, partilhar e atualizar conhecimento de forma colaborativa.



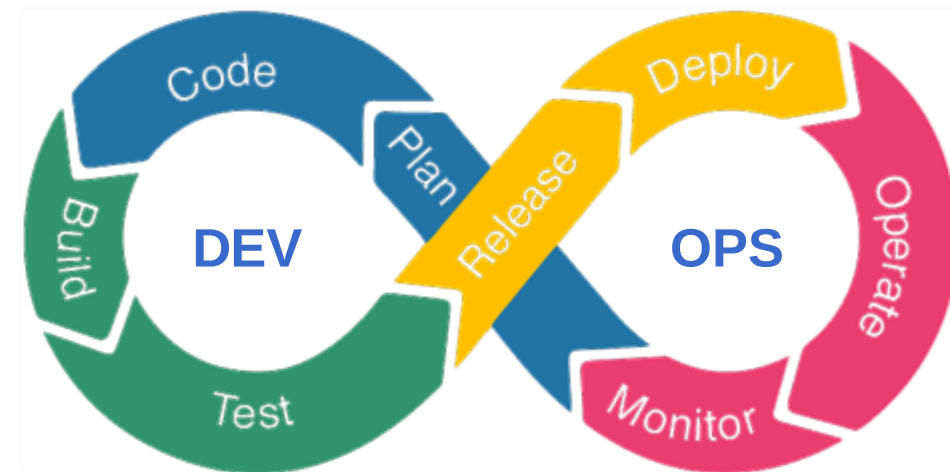
<https://github.com/til-schneider/slim-wiki>

CI/CD



A Cultura DevOps

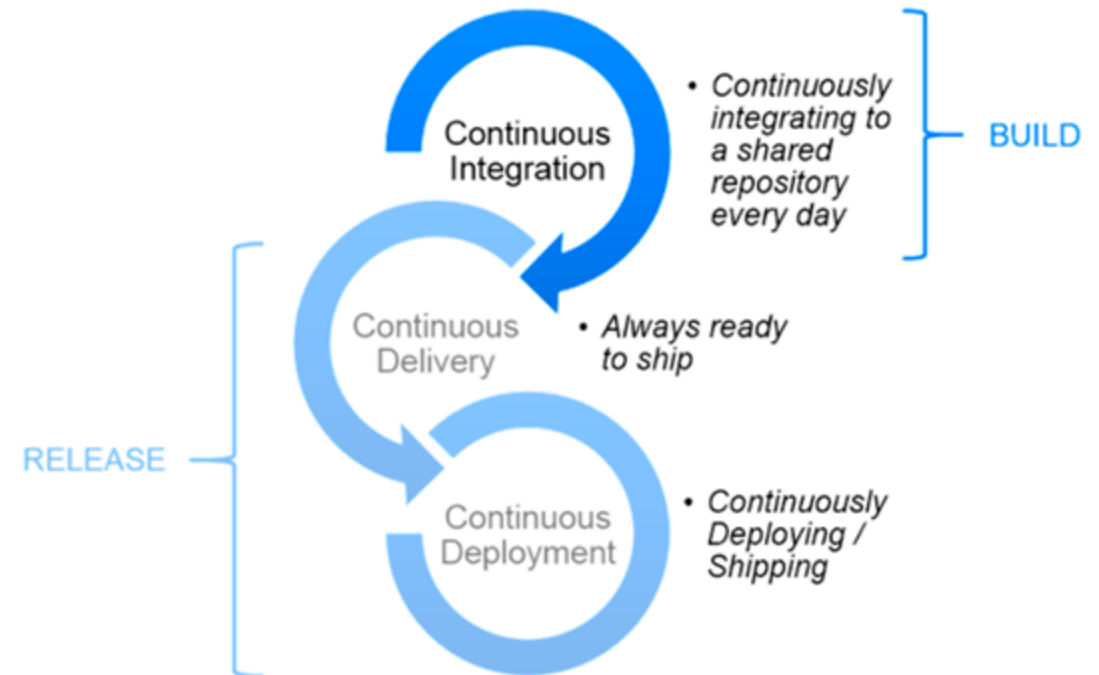
- Patrick Debois fundou o "Devopsdays" em 2008, inspirado em reuniões sobre "Infraestrutura Ágil" e uma apresentação Allspaw/Hammond.
- O DevOps promove a colaboração entre desenvolvimento e operações para acelerar a entrega de software e melhorar sua qualidade.
- Ele se concentra em comunicação, colaboração e automação.
- A adoção do DevOps implica em uma mudança cultural em direção à entrega contínua.
- O ciclo DevOps inclui planejamento, codificação, construção, teste, entrega, implantação, operação e monitoramento de software.





Integração Contínua

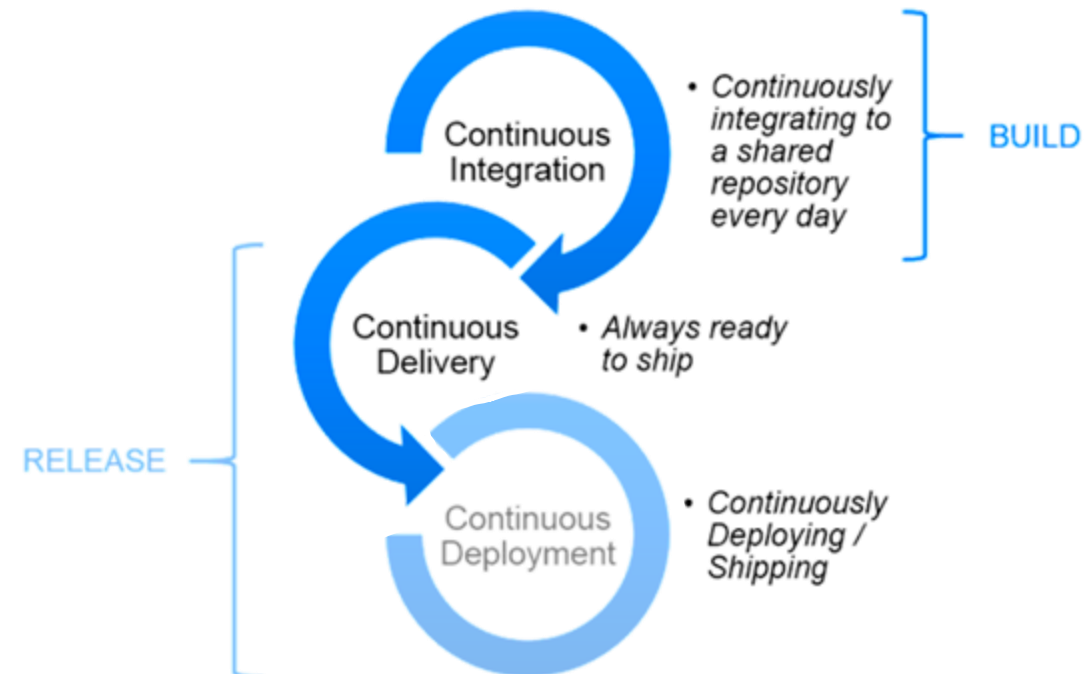
- A Integração Contínua (CI) mescla automaticamente as alterações de código em um repositório central.
- Detecta conflitos entre transferências novas e existentes.
- Executa testes automatizados para garantir a qualidade do código.
- Um “commit” no branch master aciona construção e testes automatizados.
- É usado para revisar e mesclar transferências por meio de solicitações de mesclagem.
- Contribui para a redução de custos e melhoria da qualidade do software.





Entrega Contínua

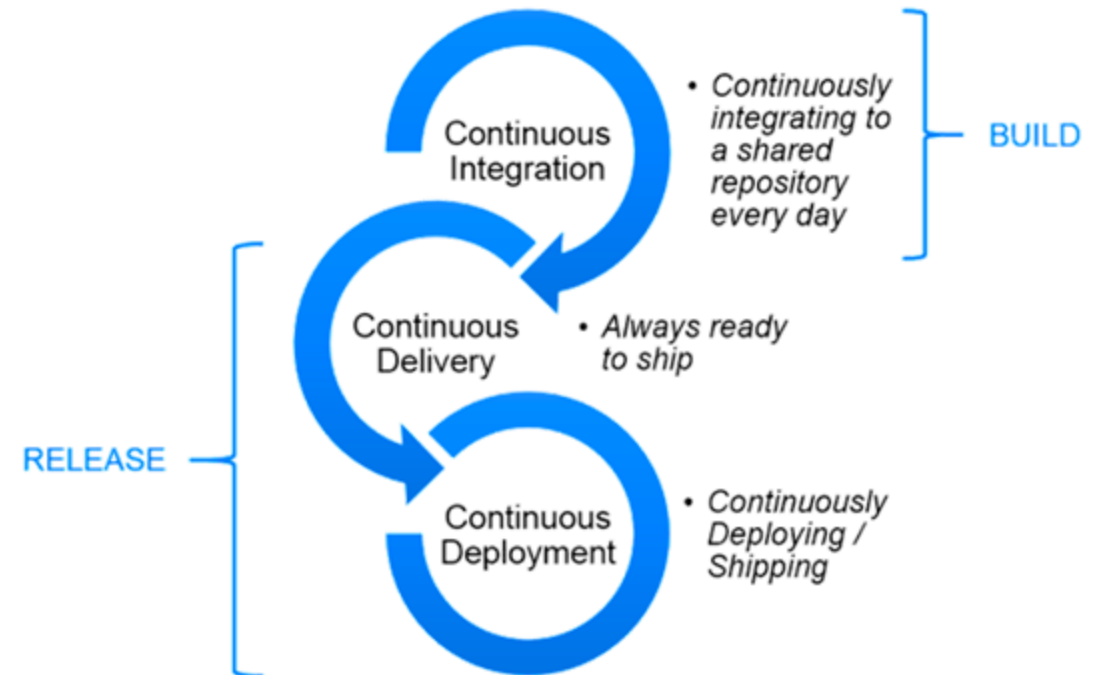
- A Entrega Contínua (CD) automatiza a preparação de alterações de código para implantação.
- Estende a Integração Contínua (CI), onde os desenvolvedores integram alterações de código com frequência.
- O objetivo é manter o código sempre implantável para implantações rápidas e confiáveis.
- Simplifica o processo de implantação, reduz erros manuais e acelera a entrega de valor.





Implantação Contínua

- Envolve a implantação automática de alterações de código na produção.
- As mudanças são implantadas assim que são concluídas e passam nos testes automatizados.
- É uma extensão da Entrega Contínua.
- Permite a entrega rápida e frequente de novos recursos e correções de bugs.
- Nenhuma intervenção humana necessária para implantação em produção.

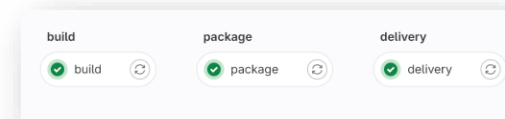




CI/CD “Pipelines”

- Automatiza fluxos de trabalho desde Integração Contínua (CI) até Entrega Contínua (CD).
- Seu objetivo é agilizar o desenvolvimento e garantir a qualidade do software.
- Implementado com linguagens específicas como Jenkinsfile, YAML, Groovy, etc.
- Suportado por ferramentas CI/CD como Jenkins, GitLab CI/CD, entre outras.

```
1 stages:
2   - build
3   - package
4   - delivery
5
6 variables:
7   APP_VERSION: "1.0.0"
8   ARTIFACT_NAME: "bank-system"
9
10 build:
11   stage: build
12   script:
13     - mvn clean install
14 package:
15   stage: package
16   script:
17     - echo "Packaging the application..."
18     - mkdir -p target
19     - cp target/build.jar $ARTIFACT_NAME.jar
20 delivery:
21   stage: delivery
22   script:
23     - echo "Delivering the application..."
24     - scp $ARTIFACT_NAME.jar $SERVER_USER@$SERVER_IP:$SERVER_PATH
25
```



Apresentação de Projetos



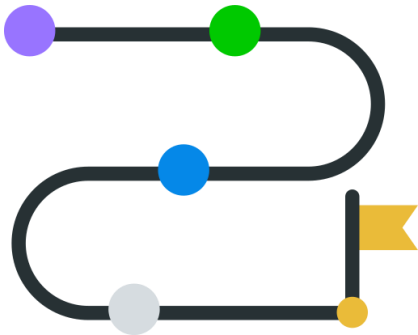
Demonstração do produto

Uma demonstração do produto mostra sua funcionalidade e como ele soluciona os problemas do cliente. Para uma demonstração adequada recomenda-se:

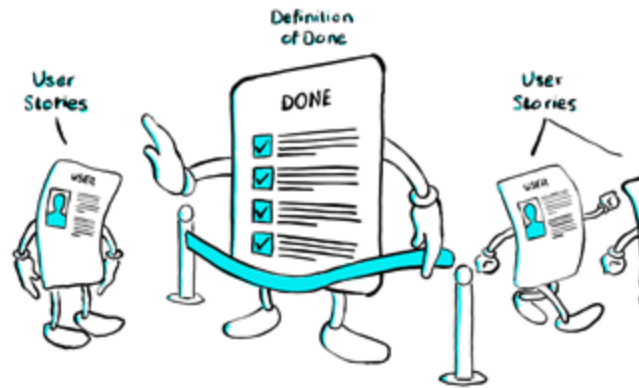
- **Conhecer o público**, uma vez que facilita a adaptação e interação do conteúdo.
- **Definir os objetivos**, uma vez que estrutura a apresentação e destaca os principais benefícios.
- **Ter uma agenda**, pois organiza a apresentação e garante a cobertura de pontos importantes.
- **Mostrar casos de uso reais**, de forma a confirmar a verdadeira utilidade do produto.
- **Evitar alterações de última hora**, pois garante estabilidade e gera confiança.



Artefactos do SDLC que se puede incluir en las demostraciones



Roadmap



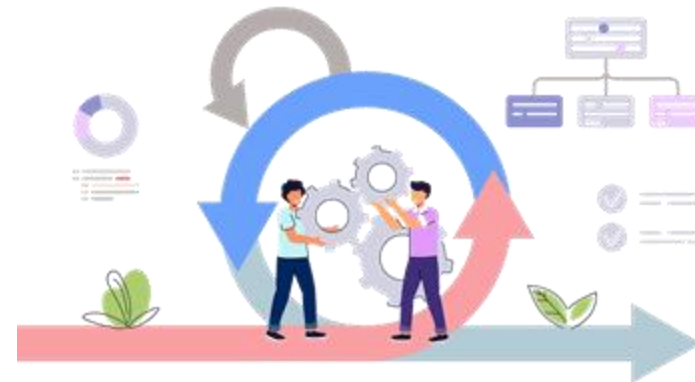
Situação das histórias de usuário



Casos de Testes e Resultados



Diagramas técnicos



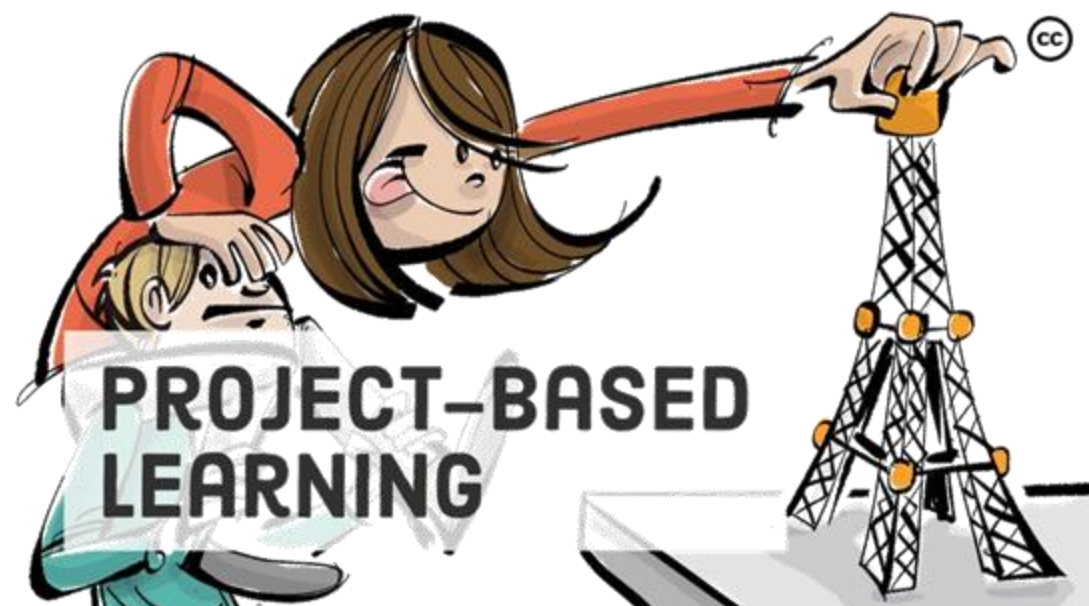
Detalhes do Processo de Desenvolvimento

Introdução ao Projeto Final

Projeto Capstone

- Introdução

- Para este curso é proposto um Projeto Capstone que será apresentado na Semana 1 e será desenvolvido ao longo de todo o curso, com entregas parciais no meio (parcial) e no final (concluído).
- Continuando com a filosofia do SD-2, o objetivo do SD-3 é avaliar (além da qualidade do código) a adoção de melhores práticas e técnicas de desenvolvimento. Portanto, serão considerados todos os artefatos resultantes da adoção do SDLC, como wikis, planejamento, diagramas técnicos, trabalho em equipe, organização, backlog, roadmap, revisões de código, modo de ramificação, etc.

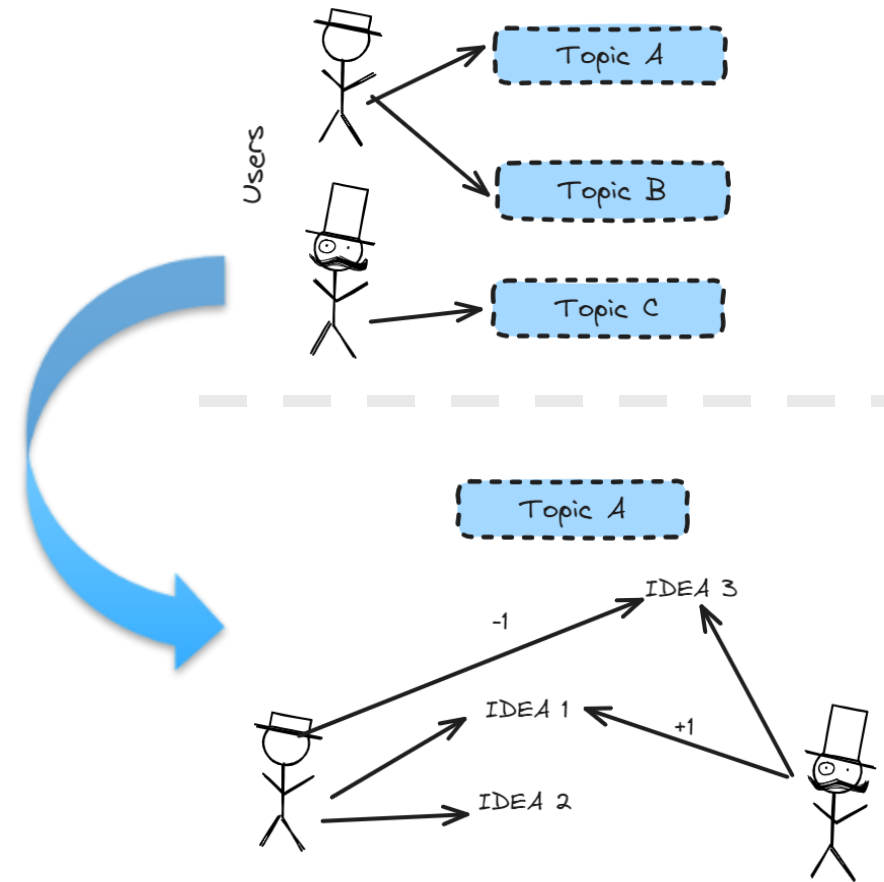




Projeto Capstone

- A definição do produto

- Capstone é baseado no desenvolvimento de uma Solução de Ideação Colaborativa ou CIS, “[Crowdsourced](#) Ideation Solution”.
- Esta solução é **uma plataforma de criatividade coletiva, onde os utilizadores podem propor ideias e também votar nas melhores, onde:**
 1. As ideias são organizadas por tópico. Os usuários podem propor tópicos arbitrários (por exemplo, "Hábitos de estudo recomendados", "Reduzir o aquecimento global", etc.).
 2. Os usuários podem então propor ideias inovadoras para esses tópicos e também votar na melhor ideia proposta.

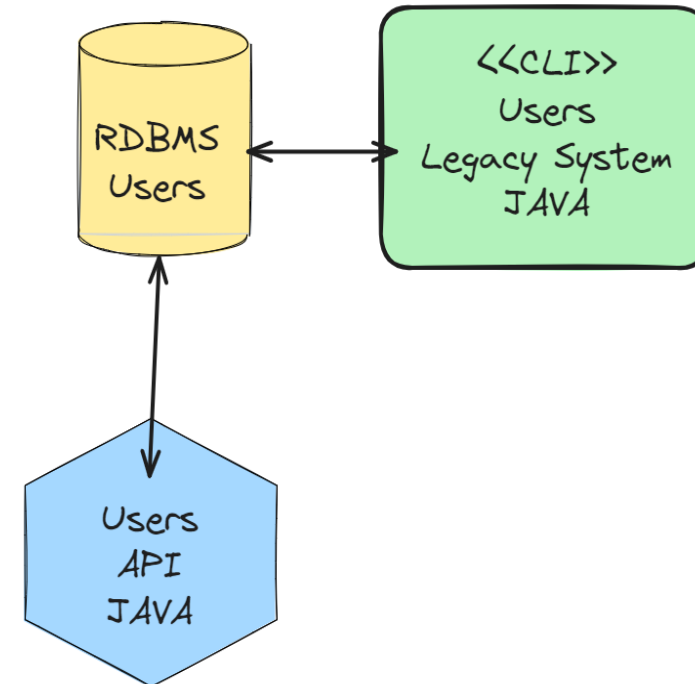




Projeto Capstone

- Fase 1: Suporte de Sistemas “Legacy”

- O CIS deve ser compatível e integrado com um sistema “Legacy” ou legado (CLI baseado em Java) e um banco de dados existente (RDBMS MySQL) que armazena dados do usuário, como identificador, login único, nome e senha.
- A solução CIS deve utilizar dados do usuário do sistema “Legacy” (Usuários cadastrados no CIS) e deve-se cuidar da consistência dos dados devido a alterações (exclusão, criação, atualizações).

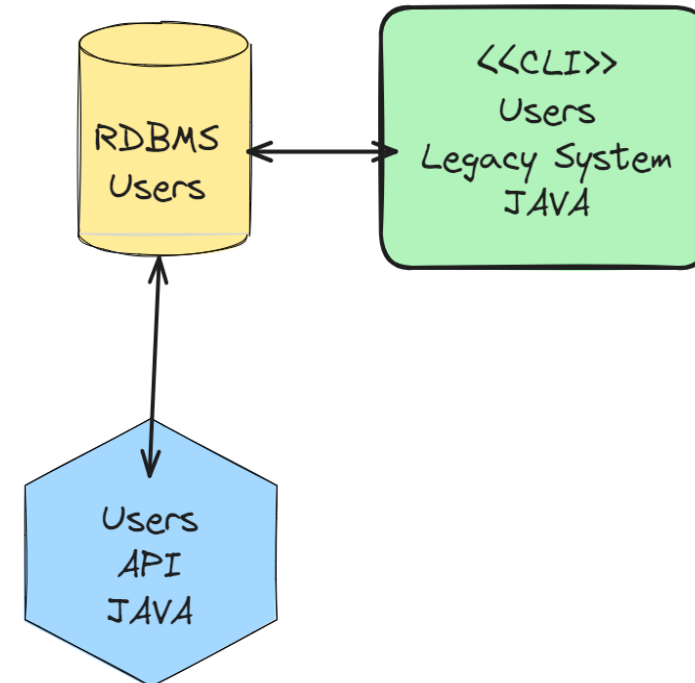




Projeto Capstone

- Fase 1: Suporte de Sistemas “Legacy”

- Uma das principais regras é que o sistema “Legacy” deve coexistir com o CIS, garantindo que ambos possam ser operáveis.
- O principal objetivo desta fase é desenvolver uma solução que atenda a essas condições seguindo um SDLC ágil e escolhendo uma estrutura de software adequada com o correspondente detalhe e qualidade.



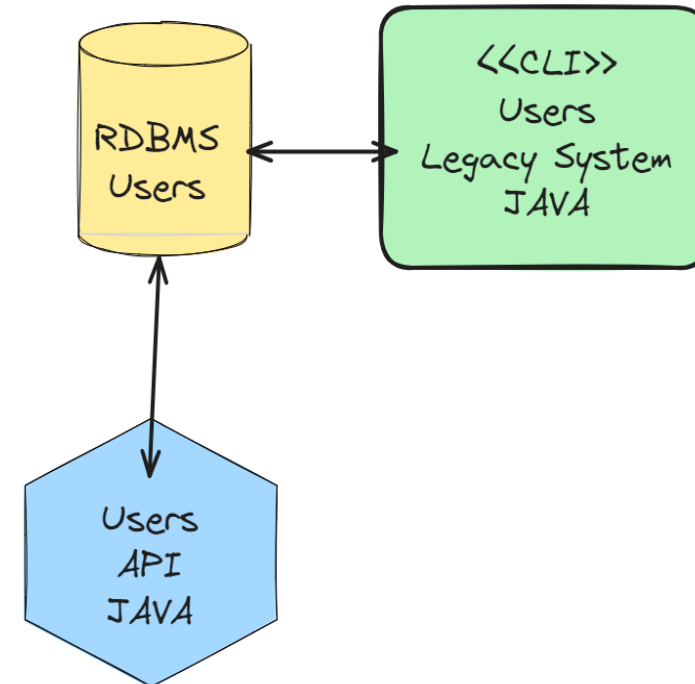


Projeto Capstone

- Fase 1: Especificações

Numa primeira fase são necessárias as seguintes funcionalidades:

- Queremos poder realizar operações CRUD nos dados do usuário com uma API de usuário moderna.
- Esta API do usuário deve ser consistente com o sistema legado em termos de dados.
- Por motivos técnicos, esta API deve ser implementada na linguagem JAVA.
- Queremos uma API cliente simples para operar e validar esta API de usuário, gostaríamos de simular o CRUD de “n” usuários.
- Esta API requer um certo nível de segurança (por exemplo, acesso autorizado com login/senha).
- Esta API deve ser definida antes de iniciar sua implementação.

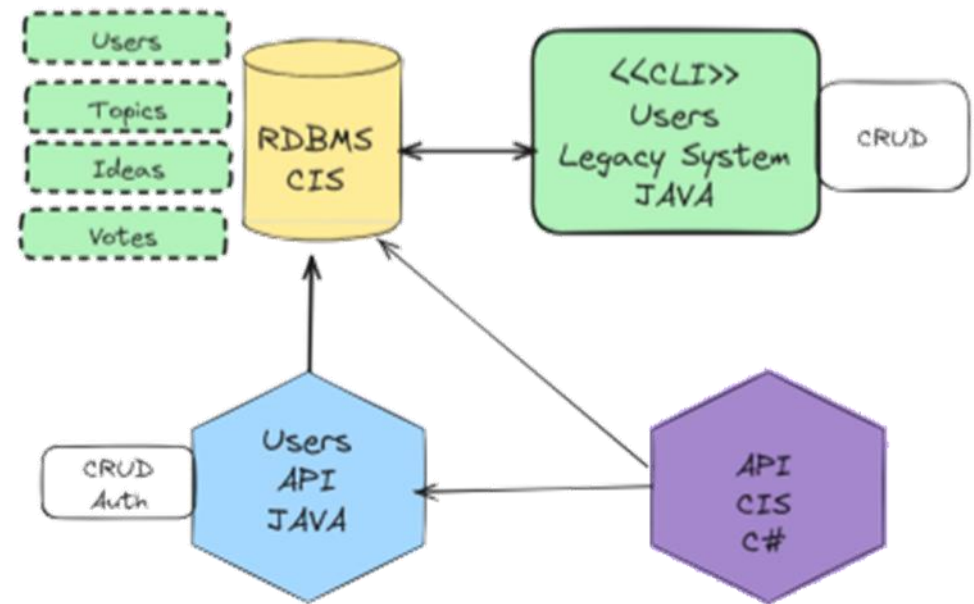




Projeto Capstone

- Fase 2: Implementação do CIS API

- É necessário um sistema que implemente a funcionalidade do CIS e se integre ao sistema legado por meio da API do usuário desenvolvida na fase 1.
- O principal objetivo desta fase é desenvolver uma solução baseada nos artefatos da fase 1, que atenda a essas condições seguindo um SDLC ágil e escolhendo a tecnologia de software adequada com o correspondente detalhamento e qualidade.



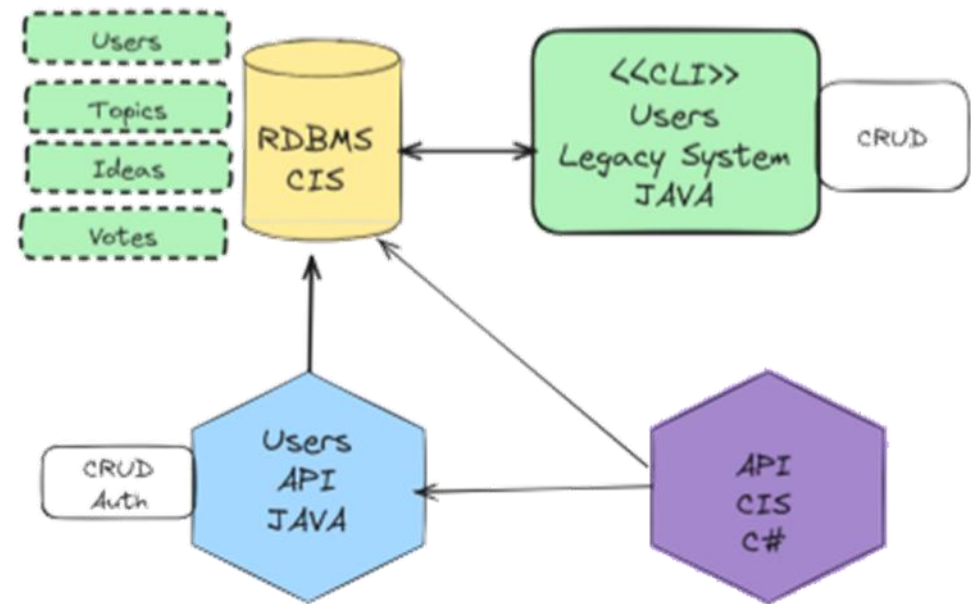


Projeto Capstone

- Fase 2: Especificações

Na segunda fase, o sistema deve atender aos seguintes requisitos:

- Precisamos manter os tópicos e ideias de nossos usuários. Portanto, precisamos estender o banco de dados existente para incluir as tabelas correspondentes.
- Queremos uma API CIS moderna que possa lidar com operações CRUD, como criação de tópicos, ideias e votação ou cancelamento de votação de ideias.
- Para usar esta API, precisamos autenticar um usuário por meio da API User.
- Queremos uma API cliente simples para operar e validar esta API CIS (simular aleatoriamente "n" usuários com tópicos, ideias, votar/cancelar votação para validar o sistema).
- Esta API deve ser definida antes de iniciar sua implementação.

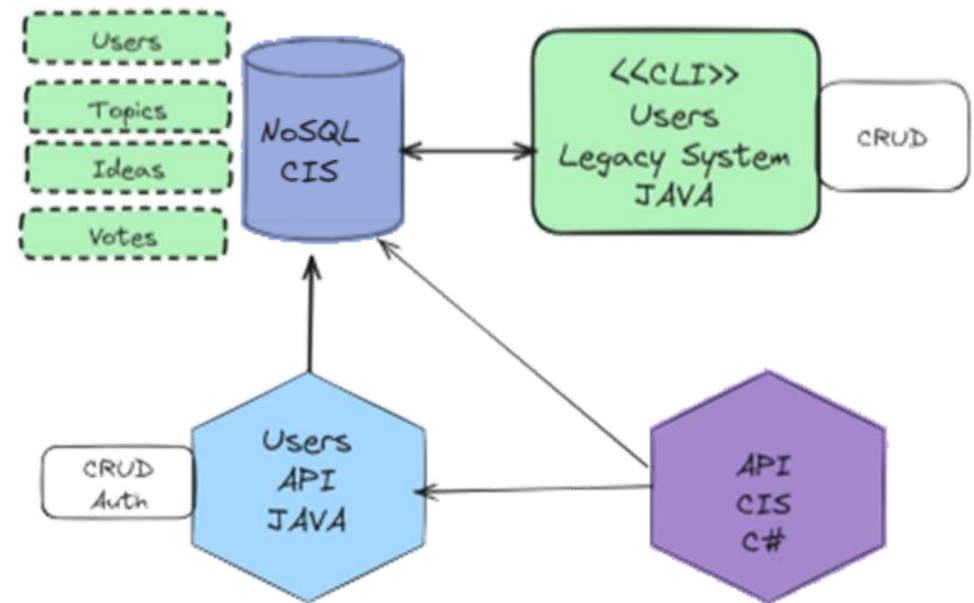




Projeto Capstone

- Fase 3: Mudanças na infraestrutura

- Devido à evolução da tecnologia, o RDBMS não pode mais ser utilizado; em vez disso, um banco de dados NoSQL (MongoDB) substituirá o banco de dados existente.
- O principal objetivo desta fase é desenvolver uma solução baseada nos artefatos da fase 2, que cumpra estas condições seguindo um ciclo de vida ágil de desenvolvimento de software, ao mesmo tempo que escolhe as tecnologias de software adequadas com o detalhe e a qualidade correspondente.



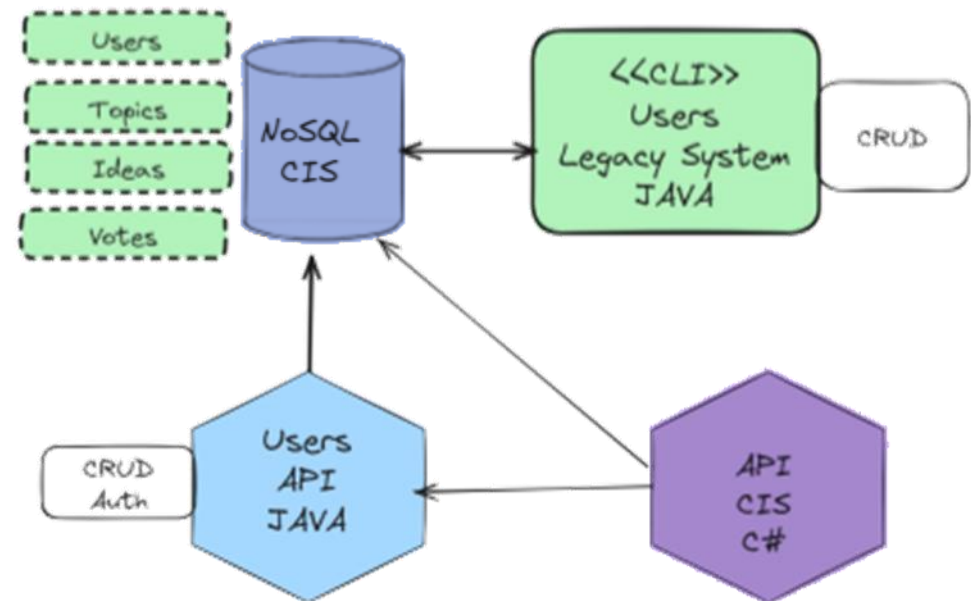


Projeto Capstone

- Fase 3: Especificações

Nesta fase, devemos atender aos seguintes requisitos:

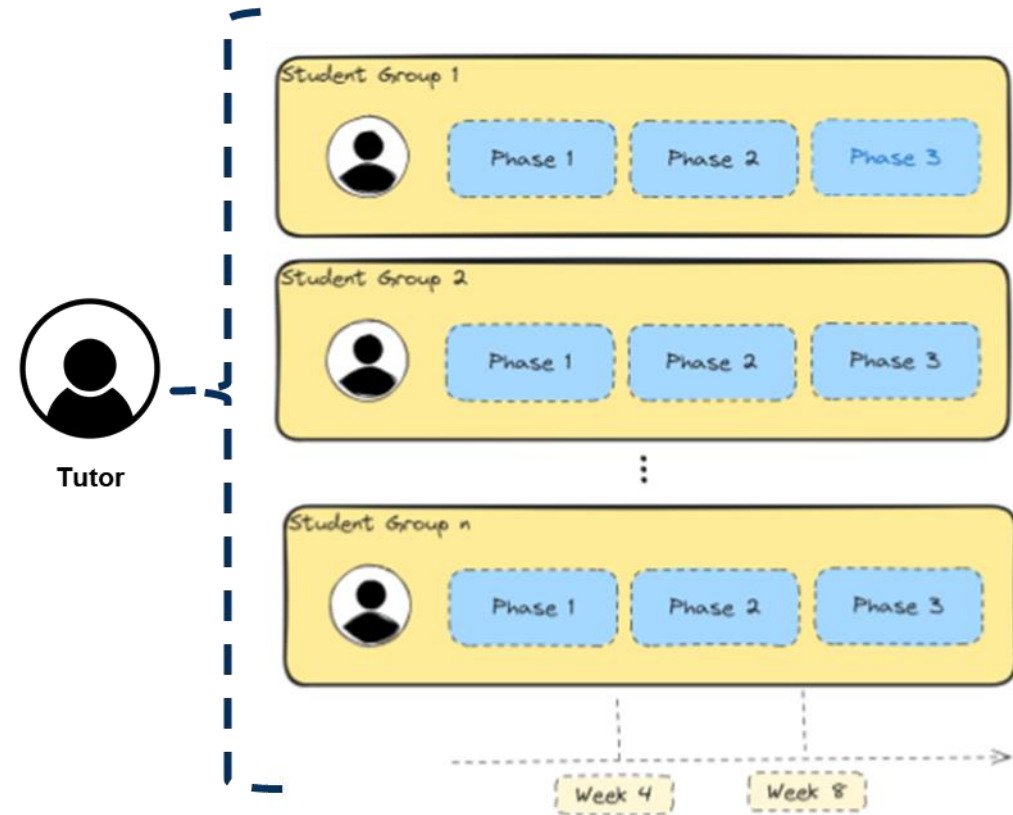
- Substitua o banco de dados existente pelo MongoDB.
- Adapte as APIs para serem compatíveis com o novo banco de dados.
- O sistema deve continuar a funcionar como antes após esta alteração do banco de dados.
- O aplicativo de linha de comando (CLI) será atualizado externamente.
- Migre os dados existentes para o banco de dados original com precisão.
- Mantenha o desempenho adequado do sistema após a movimentação.



Projeto Capstone

- Organização

- A implementação das fases do projeto será desenvolvida pelos alunos organizados em grupos (o tamanho será definido pelo tutor). O praticante organizará os alunos em equipes.
- Cada grupo aplicará um modo SDLC moderno (após as cerimônias SCRUM) para implementar cada fase como uma equipe.

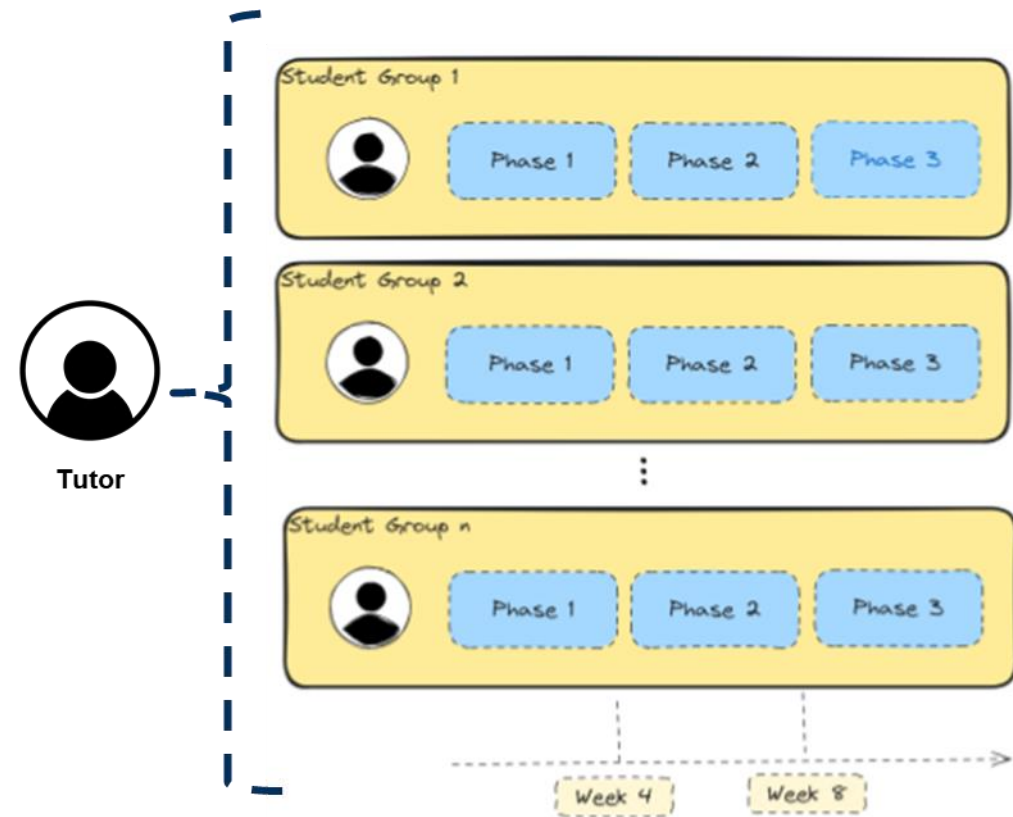




Projeto Capstone - Organização

O tutor ficará a cargo de um desses papéis:

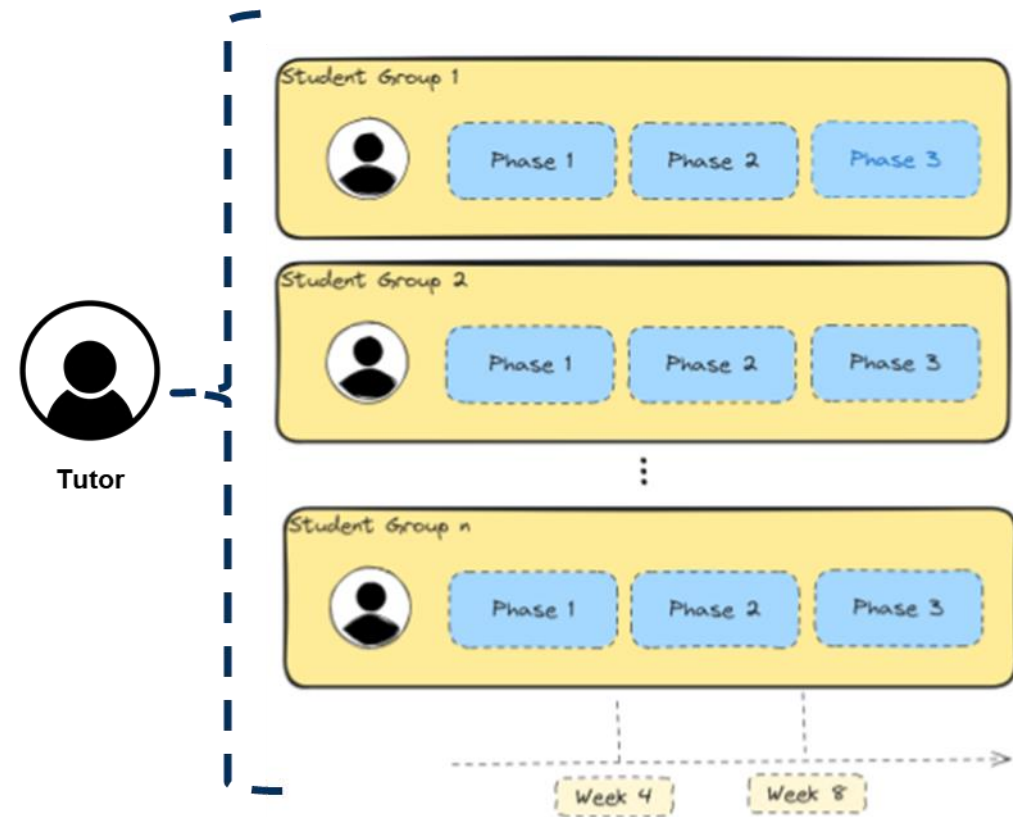
- Product Owner: Orientação sobre os objetivos de cada componente do sistema.
- Arquiteto: Será responsável pelo projeto, pelos aspectos técnicos e pela estrutura dos módulos e pela solução geral.
- Líder Técnico: Será responsável por auxiliar nas dúvidas técnicas e orientar as equipes.



Projeto Capstone - Organização

Cada equipe deve ter um Scrum Master designado para ajudar nas cerimônias, seguindo estas recomendações:

- O projeto final é a atividade principal deste curso e é também a atividade que tem maior impacto na nota.
- A cada semana, o professor explicará a parte conceitual/teórica do curso.
- Semanalmente, o aluno precisa reforçar esses conceitos com atividades práticas, neste caso, relacionadas à realização do projeto final.

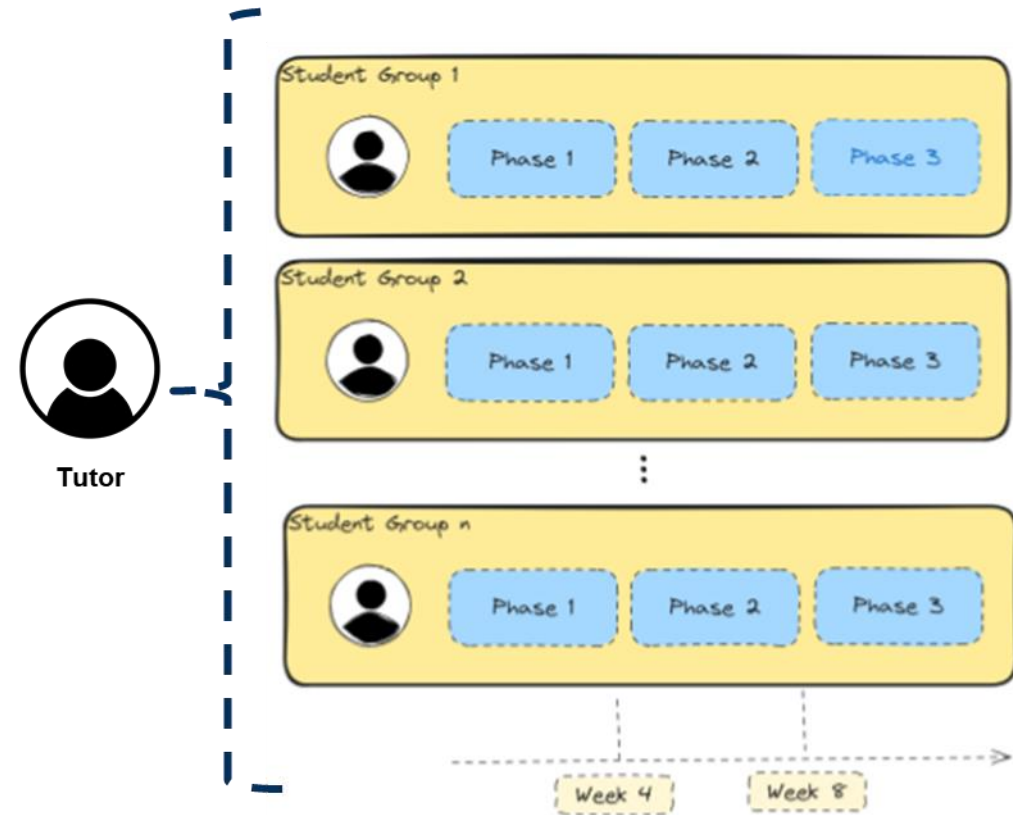


Projeto Capstone

- Organização

O desenvolvimento de cada componente da solução será avaliado em duas dimensões:

- No nível da equipe, as conquistas e o progresso da equipe como um todo (entregas, esgotamento da lista de pendências, documentação, conclusão do planejamento).
- A nível individual, as conquistas e o progresso de cada membro de uma equipe (conclusão das tarefas atribuídas, qualidade do código, etc.).
- Cada equipe deve ter um Scrum Master designado para ajudar no processo.





Projeto Capstone

- Organização

Como estamos em um conceito moderno de SDLC, a cada semana cerimônias baseadas em SCRUM devem ser articuladas em cada equipe, por exemplo:

- Semanalmente, uma sessão de planejamento é realizada por cada equipe.
- Toda semana, para cada laboratório, cada equipe realiza sua reunião ágil diária após o sprint.
- Ou o tutor ajuda com orientações sobre a forma correta de realizar as cerimônias.
- Como são muitos grupos, o tutor precisará agendar cerimônias curtas para que possa comparecer sequencialmente dentro dos laboratórios.
- Semanalmente, o tutor deve verificar a correta execução das cerimônias e os resultados corretos (artefatos SDLC, código, wikis, etc.). Se o tutor detectar problemas de qualidade ou compreensão, ele poderá optar por fornecer exercícios laboratoriais adicionais.
- A cada semana, tanto a equipe quanto os indivíduos devem ser avaliados na plataforma Canvas.

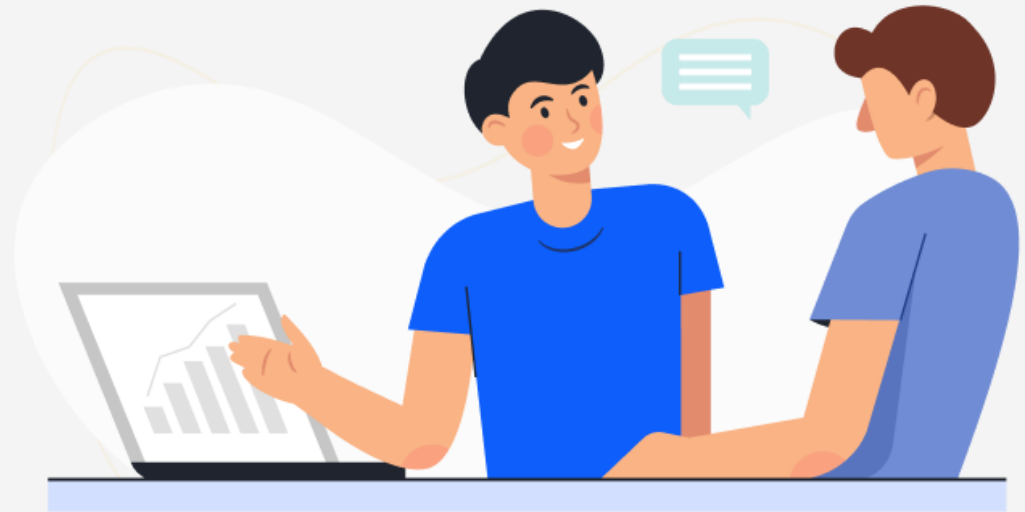
Comentários e perguntas



Entre em contato
comigo ou com
seu tutor se
precisar de
algum
esclarecimento



Reúna-se com
seus tutores esta
semana para
praticar mais e
concluir sua lição
de casa.



Obrigado pela sua atenção

